



Prof. David Fernandes de Oliveira
Instituto de Computação
UFAM

Node.js: Javascript engine

- O **Node.js** é um ambiente de execução de código JavaScript baseado na engine **V8 da Google** e na biblioteca **libuv do C++**
- De código aberto e assíncrono, proporciona um poderoso conjunto de ferramentas para criação de scripts do lado servidor

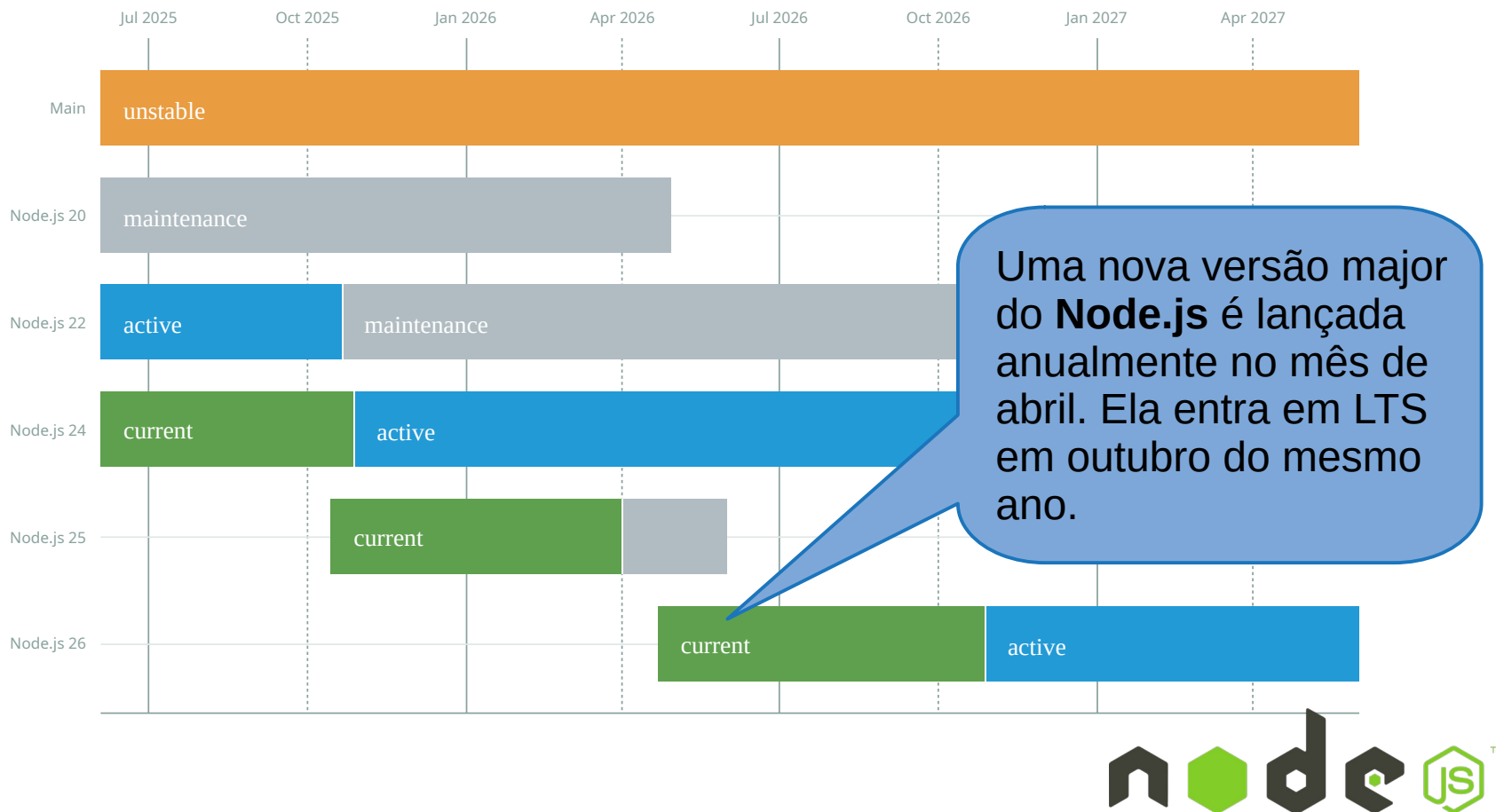


libuv



Versões do Node.js

- Atualmente, o Node.js é mantido em suas principais versões: **Active** (Long Term Support) e **Current**



Versões do Node.js

- O Node.js está disponível nos repositórios da maioria das distribuições Linux, mas pode estar desatualizado
- Para instalar a versão mais recente (LTS), recomenda-se usar o pacote **NVM**:

```
$ nvm install --lts
```

- **Node Version Manager (NVM)** é uma ferramenta que permite gerenciar várias versões do Node.js sistemas Linux
- Com o NVM, pode-se alternar entre diferentes versões do Node.js, instalar novas e remover aquelas que não precisa mais



Versões do Node.js

Baixar o Node.js®

Baixe o Node.js®

v24.14.0 (LTS) ▾

para

 Linux ▾

usando

 nvm ▾

com

 npm ▾

Informação Quer novos recursos mais cedo? Obtenha a [versão mais recente do Node.js](#) e experimente as últimas melhorias!

```
1 # Baixar e instalar o nvm:
2 curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
3
4 # Carregar o nvm sem precisar reiniciar o shell
5 \. "$HOME/.nvm/nvm.sh"
6
7 # Baixar e instalar o Node.js:
8 nvm install 24
9
10 # Verifique a versão do Node.js:
11 node -v # Deve exibir "v24.14.0".
12
13 # Verificar a versão do npm:
14 npm -v # Deve imprimir "11.9.0".
```

Bash

 Copiar para a área de transferência

"nvm" é um gerenciador de versões do Node.js multiplataforma. Se você encontrar algum problema, por favor visite [o site do nvm](#) ↗



Versões do Node.js

Baixar o Node.js®

Baixe o Node.js®

v24.14.0 (LTS) ▾

para  Linux ▾

usando

 nvm ▾

com  npm ▾

Informação Quer novos recursos mais cedo? Obtenha a [versão mais recente do Node.js](#) e experimente as últimas melhorias!

```
david@coyote:~  
david ~ $ nvm install --lts  
Installing latest LTS version.  
Downloading and installing node v24.18.0...  
Downloading https://nodejs.org/dist/v24.18.0/node-v24.18.0-linux-x64.tar.xz...  
##### 100.0%  
Computing checksum with sha256sum  
Checksums matched!  
Now using node v24.18.0 (npm v11.16.0)  
david ~ $  
took 10s Node 24.18.0
```

Bash

 Copiar para a área de transferência

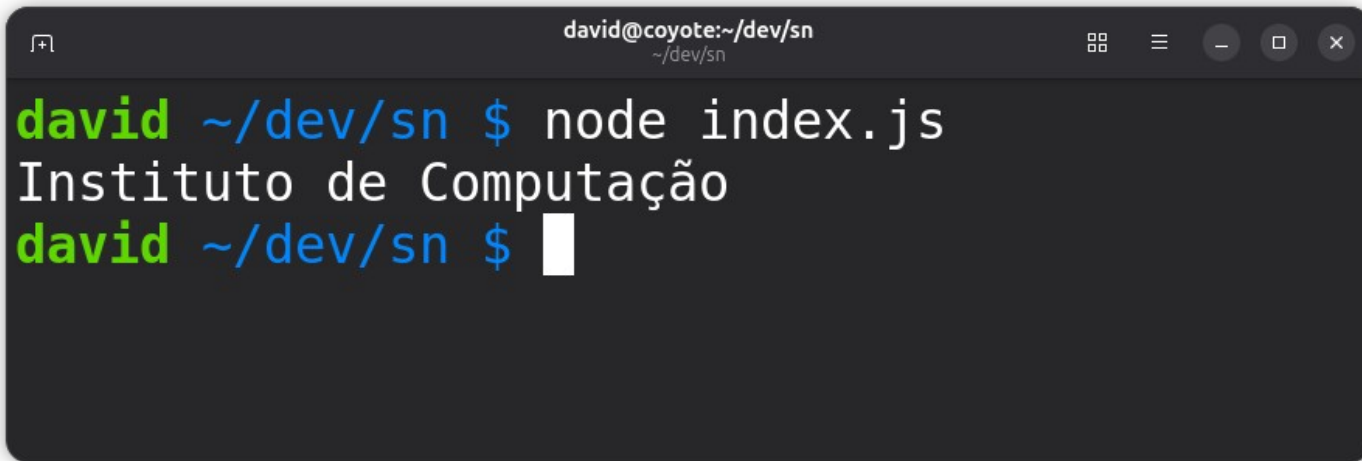
"nvm" é um gerenciador de versões do Node.js multiplataforma. Se você encontrar algum problema, por favor visite [o site do nvm](#) ➞



Executando Código JavaScript

- Para executar um código JavaScript usando **node.js** basta usar o intepretador **node** instalado no sistema operacional

```
// arquivo index.js  
console.log("Instituto de Computação")
```



A terminal window with a dark background. The title bar shows 'david@coyote:~/dev/sn' and standard window controls. The prompt is 'david ~/dev/sn \$'. The command 'node index.js' has been entered and executed. The output 'Instituto de Computação' is displayed on the next line. The prompt 'david ~/dev/sn \$' is shown again with a cursor.

```
david@coyote:~/dev/sn  
david ~/dev/sn $ node index.js  
Instituto de Computação  
david ~/dev/sn $
```



Módulos Embarcados do Node

- O Node.js é um interpretador JavaScript leve, cujos módulos embarcados (**built-in** ou **core modules**) provêem apenas funcionalidades básicas para o programador
 - Os módulos embarcados são carregados automaticamente assim que o processo Node.js inicia
 - Mesmo assim, é necessário importar (via **require** ou **import**) tais módulos antes de usá-los em sua aplicação:

```
const http = require('http');
```

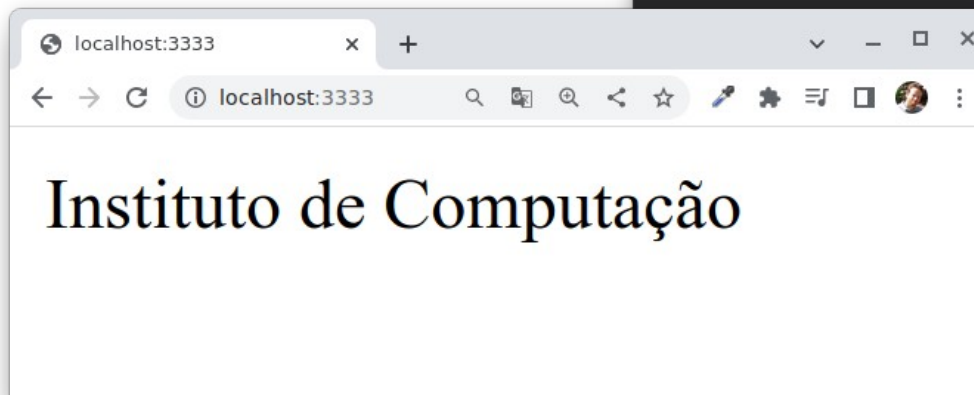
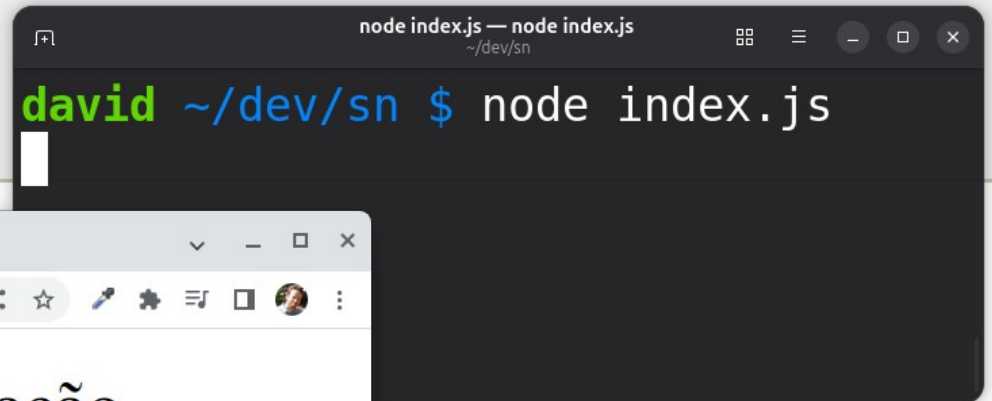
- Alguns módulos embarcados do Node.js: **http**, **url**, **path**, **fs**, **os**, **sys**, **tty**, **cluster**, **process**, **timers** e **util**



Hello World Cliente Servidor

- Também podemos gerar conteúdo que poderá ser acessado pelo browser usando o protocolo HTTP

```
const http = require('http');  
  
const server = http.createServer(function(req, res){  
  res.writeHead(200, {"Content-Type": "text/html; charset=utf-8"});  
  res.write("Instituto de Computação");  
  res.end();  
});  
  
server.listen(3333);
```



Hello World Cliente Servidor

- Também podemos gerar conteúdo que poderá ser acessado pelo browser usando o protocolo HTTP

```
const http = require('http');  
  
const server = http.createServer((res){  
  res.writeHead(200, {  
    'Content-Type': 'text/html; charset=utf-8'});  
  res.write("Instituto de Computação");  
  res.end();  
});  
  
server.listen(3333);
```

http é um módulo do NodeJS

Os módulos podem ser importados através da função **require**



O que é um Framework Web

- Um **framework Web** é um conjunto de bibliotecas que visam facilitar o desenvolvimento de sistemas Web completos
- Desenvolver uma aplicação Web do zero é muitas vezes inviável e bastante trabalhoso
 - O uso de bibliotecas e frameworks de código aberto torna tudo mais fácil, rápido, confiável, e provavelmente mais seguro

METEOR

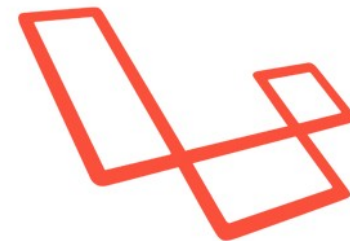
express

sails



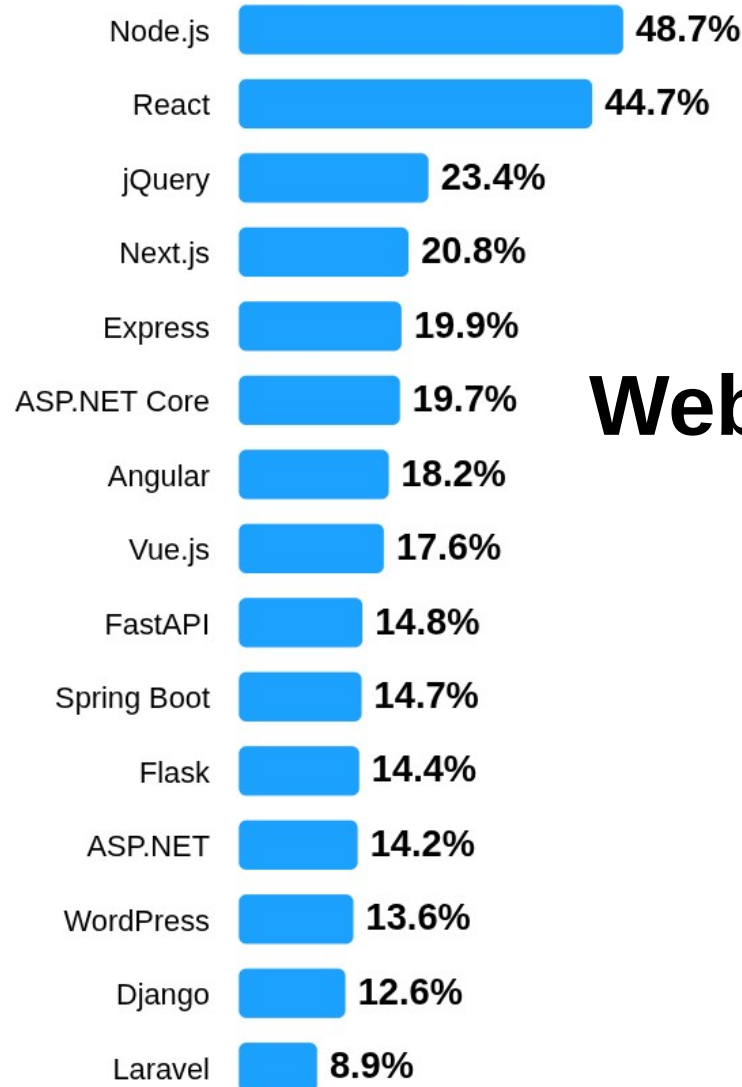
yii

django



laravel

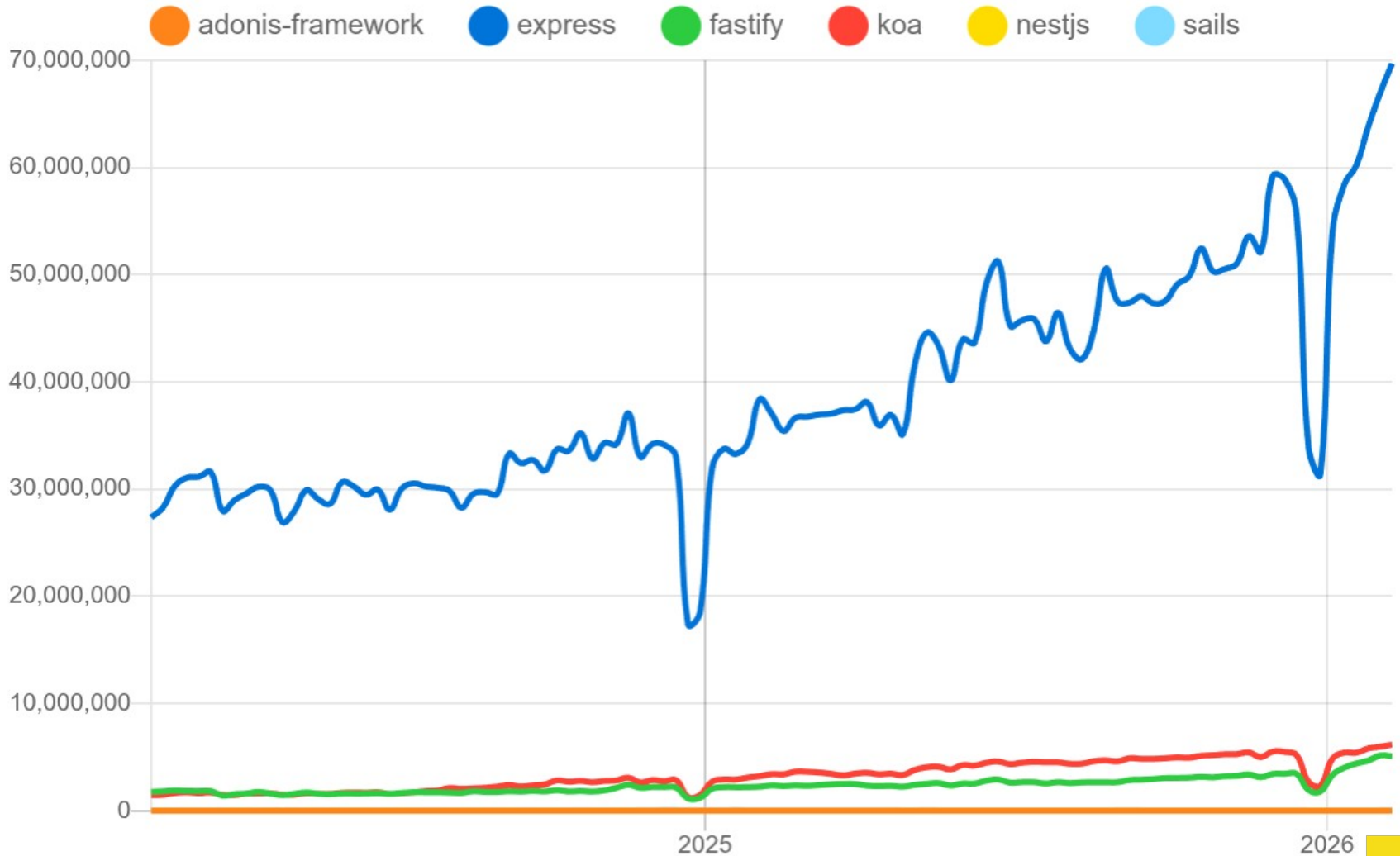
StackOverflow Surveys



Web Frameworks and Technologies 2025

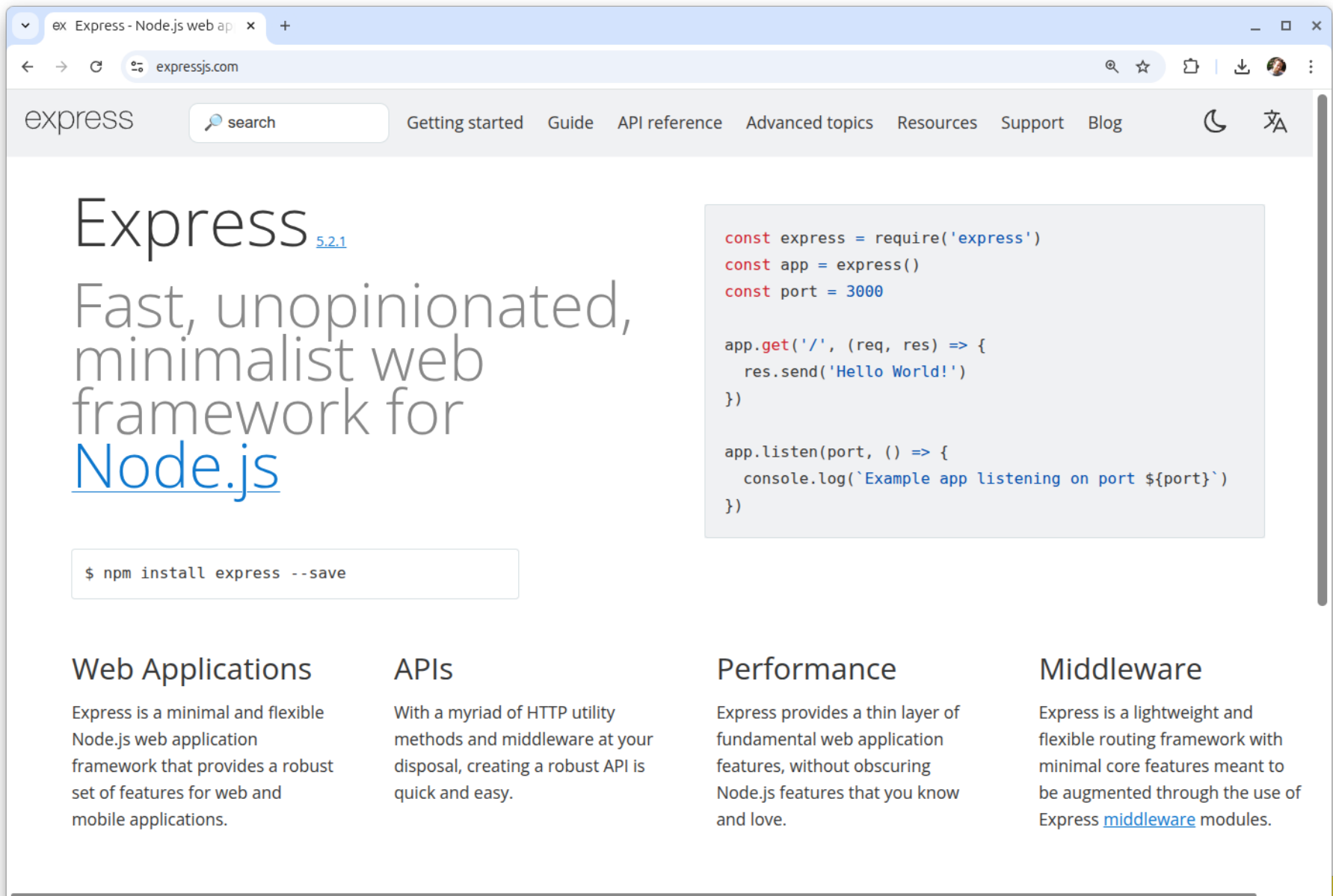


NPM Trends



Um Framework não opinativo

- O Express é um framework Web baseado no Node.js que facilita o desenvolvimento de Apps e adiciona novos recursos
- O Express segue uma filosofia **não opinativa** e **minimalista**
 - **Não opinativa** significa que ele te dá total liberdade sobre como organizar seu código dentro de sua aplicação
 - **Minimalista** significa que ele te dá total liberdade de escolher outros módulos para completar as necessidades de sua aplicação
- Essas características nos permitem usar o Express para desenvolver qualquer tipo de aplicação, de um hub de vídeos à um chat



Express [5.2.1](#)

Fast, unopinionated,
minimalist web
framework for
[Node.js](#)

```
$ npm install express --save
```

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Web Applications

Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

APIs

With a myriad of HTTP utility methods and middleware at your disposal, creating a robust API is quick and easy.

Performance

Express provides a thin layer of fundamental web application features, without obscuring Node.js features that you know and love.

Middleware

Express is a lightweight and flexible routing framework with minimal core features meant to be augmented through the use of Express [middleware](#) modules.

O Arquivo package.json

- O primeiro passo no desenvolvimento de uma aplicação node.js é a criação de um arquivo chamado **package.json**
 - A função desse arquivo é armazenar vários metadados sobre a aplicação, incluindo suas dependências

```
{  
  "name": "hello-world",  
  "author": "David Fernandes",  
  "private": true,  
  "version": "0.0.1",  
  "dependencies": {}  
}
```

O Arquivo package.json

- O primeiro passo no desenvolvimento de uma aplicação node.js é a criação de um arquivo chamado **package.json**
 - A função desse arquivo é armazenar vários metadados sobre a aplicação, incluindo suas dependências

```
{  
  "name": "hello-world",  
  "author": "David Fernandes",  
  "private": true,  
  "version": "0.0.1",  
  "dependencies": {}  
}
```

Para criar um novo arquivo **package.json** com os valores desejados, use o comando **npm init**. Esse comando fará algumas perguntas para o programador, e criará um arquivo **package.json** baseado nas suas respostas.

O Arquivo package.json

- Outra opção é executar o comando **npm init** com a opção **-y**, que irá criar um package.json com valores iniciais

```
david@coyote:~/dev/sn
~/dev/sn
david ~/dev/sn $ npm init -y
Wrote to /home/david/dev/sn/package.json:

{
  "name": "sn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Adicionando o Express no Projeto

- Para incluirmos o framework express no projeto, basta executarmos o comando **npm install express**
 - O pacote express é adicionado automaticamente como uma dependência do projeto

```
david@coyote:~/dev/sn
~ /dev/sn

david ~/dev/sn $ npm install express

added 67 packages, and audited 68 packages in 4s

26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
david ~/dev/sn $
```

took 4s

Adicionando o Express no Projeto

- Para incluirmos o framework express no projeto, basta

```

david ~/dev/sn $ cat package.json
{
  "name": "sn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}
david ~/dev/sn $
```


Semantic Versioning - SemVer

- O **Versionamento Semântico** (SemVer) é uma convenção para indicar o impacto das alterações realizadas em um software
- Uma versão SemVer contém 3 números: MAJOR.MINOR.PATCH
 - MAJOR: alterações incompatíveis com versões anteriores
 - MINOR: novas funcionalidades compatíveis com versões anteriores
 - PATCH: correções de bugs compatíveis com versões anteriores
- Exemplo de evolução:
 - 5.1.0 → 5.1.1 correções de bug
 - 5.1.1 → 5.2.0 novas funcionalidades
 - 5.2.0 → 6.0.0 alterações incompatíveis

Versionamento Semântico 2.0.0

Sumário

Dado um número de versão MAJOR.MINOR.PATCH, incremente a:

1. versão Maior(MAJOR): quando fizer mudanças incompatíveis na API,
2. versão Menor(MINOR): quando adicionar funcionalidades mantendo compatibilidade, e
3. versão de Correção(PATCH): quando corrigir falhas mantendo compatibilidade.

Rótulos adicionais para pré-lançamento(pre-release) e metadados de construção(build) estão disponíveis como extensão ao formato MAJOR.MINOR.PATCH.

Introdução

No mundo de gerenciamento de software existe algo terrível conhecido como inferno das dependências ("dependency hell"). Quanto mais o sistema cresce, e mais pacotes são adicionados a ele, maior será a possibilidade de, um dia, você encontrar-se neste poço de desespero.

Em sistemas com muitas dependências, lançar novos pacotes de versões pode se tornar rapidamente um pesadelo. Se as especificações das dependências são muito amarradas você corre o risco de um bloqueio de versão (A falta de capacidade de atualizar um pacote sem ter de liberar novas versões de cada pacote dependente). Se as dependências são vagamente especificadas, você irá inevitavelmente ser mordido pela 'promiscuidade da versão' (assumindo compatibilidade com futuras versões mais do que

Semantic Versioning - SemVer

- O Express utiliza versionamento semântico para numerar suas versões – por exemplo: **express@5.2.1**
- O **package.json** normalmente registra algo semelhante a:

```
{  
  "dependencies": {  
    "express": "^5.2.1"  
  }  
}
```

- Ao executarmos o comando **npm update**, o símbolo ^ permite que o npm instale versões dentro da mesma versão MAJOR
- Desta forma, o ^5.2.1 permite a instalação de versões 5.2.2, 5.3.0 e 5.5.3, mas não permite 6.0.0

Semantic Versioning - SemVer

- No **package.json**, se trocarmos o ^ por ~, então as atualizações automáticas ficarão restritas à novos PATCHs

```
{
  "dependencies": {
    "express": "~5.2.1"
  }
}
```

- Nesse caso, o npm pode instalar versões como 5.2.2, 5.2.3 e 5.2.9, mas não instalará automaticamente 5.2.0 e 6.0.0
- Desta forma, o uso do símbolo ~ ao invés de ^ deixa a atualização das dependências **mais conservadora**

Instalação do TypeScript

- Após a instalação do Express, vamos instalar o **TypeScript** como dependência de desenvolvimento da aplicação
- Também serão instalados os pacotes de declaração **@types** para **Express** e **Node.js**, que fornecem definições de tipos
- Além disso, o pacote **tsx** será útil para executar código Typescript

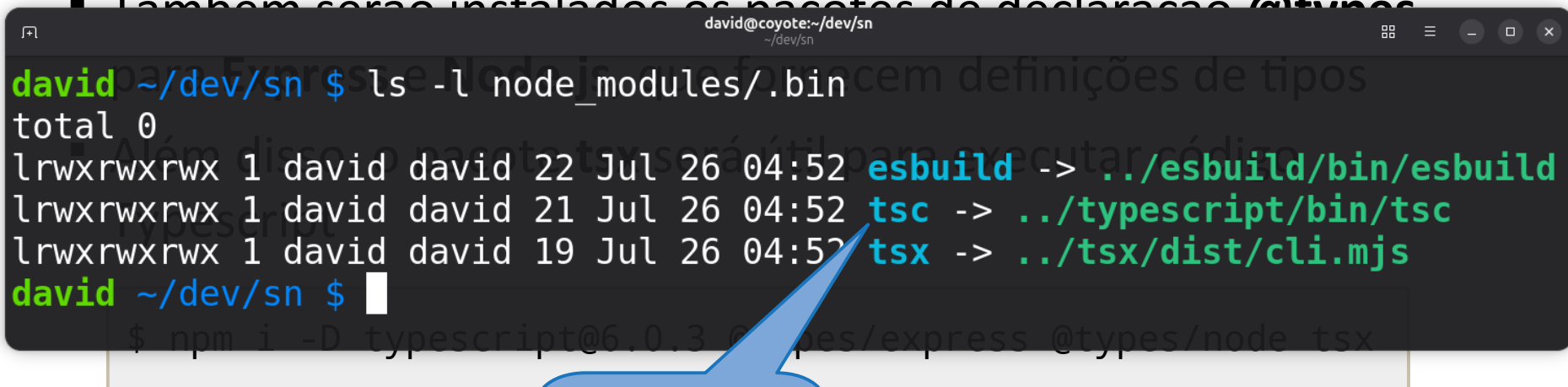
```
$ npm i -D typescript@6.0.3 @types/express @types/node tsx
```

- A opção **-D**, ou **--save-dev**, faz com que o npm instale esses pacotes como dependências de desenvolvimento

Instalação do TypeScript

- Após a instalação do Express, vamos instalar o **TypeScript** como dependência de desenvolvimento da aplicação

- Também serão instalados os pacotes de declaração **@types**



A terminal window with a dark background. The prompt is 'david@coyote:~/dev/sn'. The user runs 'ls -l node_modules/.bin' and the output shows three files: 'esbuild', 'tsc', and 'tsx'. Each file has permissions 'lrwxrwxrwx', owner 'david', group 'david', and a date '22 Jul 26 04:52'. The paths are './esbuild/bin/esbuild', './typescript/bin/tsc', and './tsx/dist/cli.mjs'. Below this, the user runs '\$ npm i -D typescript@6.0.3 @types/express @types/node tsx'.

```
david@coyote:~/dev/sn
david ~/dev/sn $ ls -l node_modules/.bin
total 0
lrwxrwxrwx 1 david david 22 Jul 26 04:52 esbuild -> ../esbuild/bin/esbuild
lrwxrwxrwx 1 david david 21 Jul 26 04:52 tsc -> ../typescript/bin/tsc
lrwxrwxrwx 1 david david 19 Jul 26 04:52 tsx -> ../tsx/dist/cli.mjs
david ~/dev/sn $
$ npm i -D typescript@6.0.3 @types/express @types/node tsx
```

- A opção **-D**, ou **--save-dev**, indica que o npm instale esses pacotes como dependência de desenvolvimento

Compilador
TypeScript

Instalação do TypeScript

- Agora podemos iniciar as configurações do TypeScript, através do seguinte comando:

```
$ npx tsc --init
```

- O comando acima irá criar um arquivo **tsconfig.json**, que contém opções padrão do compilador TypeScript e nos permite ajustar ou personalizar essas opções



```
david@coyote:~/dev/sn  
david ~/dev/sn $ npx tsc --init  
Created a new tsconfig.json  
You can learn more at https://aka.ms/tsconfig  
david ~/dev/sn $
```

A terminal window with a dark background. The title bar shows 'david@coyote:~/dev/sn'. The prompt is 'david ~/dev/sn \$'. The command 'npx tsc --init' is entered. The output is 'Created a new tsconfig.json' followed by 'You can learn more at https://aka.ms/tsconfig'. The prompt returns to 'david ~/dev/sn \$'. A blue square with the text 'TS' is positioned to the right of the output text.

Instalação do TypeScript

- No **tsconfig.json** gerado, descomente as seguintes linhas:

```
"rootDir": "./src",  
"outDir": "./dist",
```

- No **package.json** gerado, troque o valor de **type** para **module**

```
"type": "module",
```

Hello World em Express/TypeScript

- Para fazer um primeiro teste com o Express/TypeScript, crie o arquivo **src/index.ts** e adicione o seguinte conteúdo:

```
import express, { type Request, type Response } from "express";  
const app = express();  
const PORT = 3333;  
  
app.get("/", (req: Request, res: Response) => {  
  res.send("Hello world!");  
});  
  
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}.`);  
});
```

Inicializa o
servidor HTTP
na porta
PORT

Hello World em Express/TypeScript

- Para fazer um primeiro teste com o Express, vamos criar o arquivo **src/index.ts** e adicionar o seguinte código. Para rodar o código, podemos usar o pacote **tsx** instalado no projeto. Para isso, crie o arquivo **src/index.ts** e adicione o seguinte código.

Para rodar o código, podemos usar o pacote **tsx** instalado no projeto.

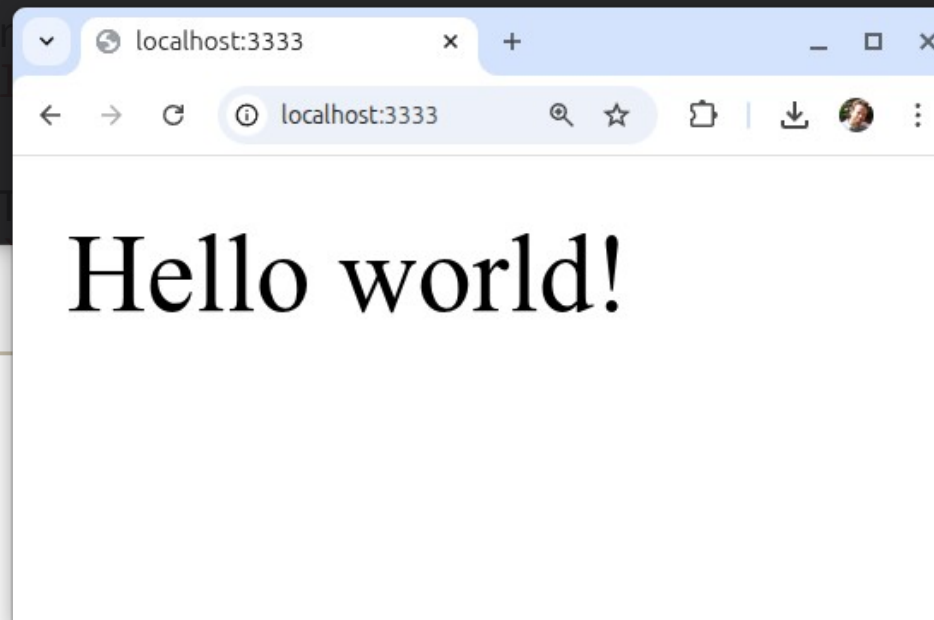
Inicializa o servidor HTTP na porta **PORT**.

```
import express, { Request, Response } from 'express';
const app = express();
const PORT = 3333;

app.get('/', (req, res) => {
  res.send('Hello world!');
});

app.listen(PORT, () => {
  console.log(`Server running on port 3333.`);
});
```

```
david ~/dev/sn $ npx tsx src/index.ts
Server running on port 3333.
```



Variáveis de Ambiente

- Variáveis de ambiente são dados de configuração que ficam disponíveis no ambiente de execução dos programas
- No Node, as variáveis de ambiente constituem uma ótima maneira de definir as configurações locais de um programa
 - Como URLs, portas, chaves de autenticação, senhas, etc
- Por exemplo, para criar uma variável de ambiente para armazenar a porta do serviço http:

```
process.env.DB_PORT=3333
```

Por convenção,
as variáveis de
ambiente são
escritas em
caixa alta



Variáveis de Ambiente

- Uma opção para definir as variáveis de ambiente é através de arquivos não versionados no diretório raiz da aplicação
 - Exemplos de nomes para esses arquivos são **.env**, **.env.development**, **.env.production**

```
david@coyote:~/dev/sn
david ~/dev/sn $ echo "PORT=3333" > .env
david ~/dev/sn $ cat .env
PORT=3333
david ~/dev/sn $
```

Variáveis de Ambiente

- Para que as variáveis de ambiente sejam carregadas na aplicação, podemos usar pacotes como o **dotenv**

```
david@coyote:~/dev/sn
david ~/dev/sn $ npm install dotenv
added 1 package, and audited 85 packages in 1s
27 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
david ~/dev/sn $
```

Variáveis de Ambiente

- Para que as variáveis de ambiente sejam carregadas na aplicação, podemos usar pacotes como o **dotenv**

```
import express, { type Request, type Response } from "express";
import dotenv from "dotenv";

dotenv.config({ quiet: true });

const app = express();
const PORT = process.env.PORT || 3000;

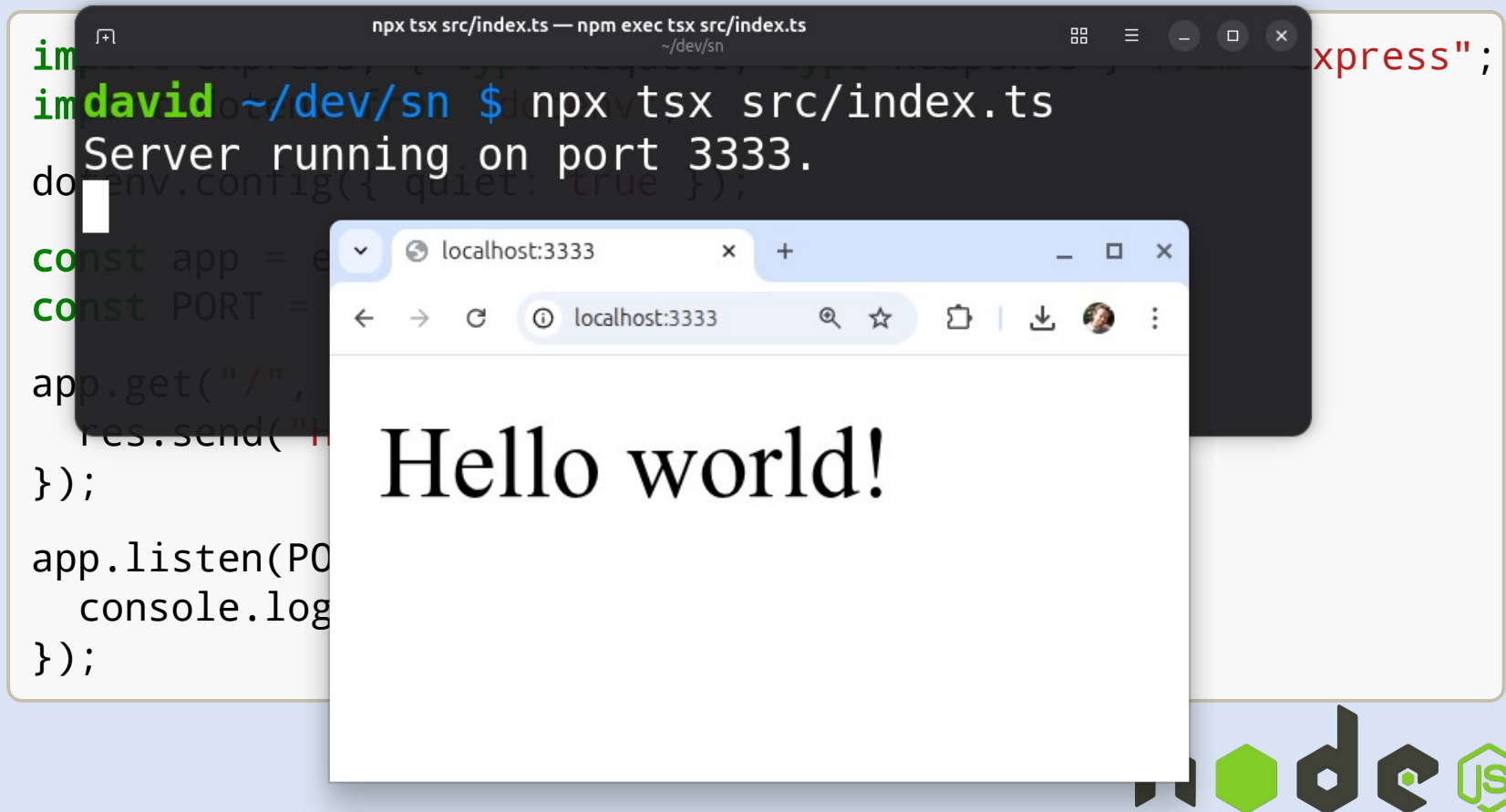
app.get("/", (req: Request, res: Response) => {
  res.send("Hello world!");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}.`);
});
```



Variáveis de Ambiente

- Para que as variáveis de ambiente sejam carregadas na aplicação, podemos usar pacotes como o **dotenv**



Variáveis de Ambiente

- As arquivos **.env** devem ser adicionados no **.gitignore**, pois as variáveis de ambiente estão relacionadas com o ambiente do usuário que está rodando a aplicação
- No entanto, quando um novo desenvolvedor faz o clone do repositório, é importante que ele saiba o nome das variáveis que ele precisa definir em seu ambiente
- Para resolver isso, pode-se criar arquivos versionáveis contendo variáveis de ambientes de exemplo
 - Por exemplo, tais arquivos podem ter nomes como **.env.example**



Validando as variáveis de ambiente

- Esquecer de adicionar uma variável de ambiente ou não usar o tipo de dado certo pode levar a erros inesperados
- Para evitar esses problemas, podemos usar o pacote **envalid** para validar as variáveis de ambiente do arquivo **.env**

```
$ npm i envalid
```

Validando as variáveis de ambiente

- Para usar o **envalid**, vamos criar um diretório **utils** no diretório **src** e adicionar um arquivo chamado **getEnv.ts**

```
david@coyote:~/dev/sn/src $ tree
.
├── index.ts
└── utils
    └── getEnv.ts

2 directories, 2 files
david ~/dev/sn/src $
```

Validando as variáveis de ambiente

- No arquivo **getEnv.ts**, definimos todas as variáveis de ambiente necessárias para rodar a aplicação
- O **envalid** lançará um erro caso esqueçamos de fornecer alguma dessas variáveis no **.env** ou se forem do tipo errado

```
import { cleanEnv, str, port } from "envalid";
import dotenv from "dotenv";

dotenv.config({ quiet: true });

export function getEnv() {
  return cleanEnv(process.env, {
    NODE_ENV: str(),
    PORT: port({ default: 3000 }),
  });
}
```

envalid - npm

npmjs.com/package/envalid

Validator types

Node's `process.env` only stores strings, but sometimes you want to retrieve other types (booleans, numbers), or validate that an env var is in a specific format (JSON, url, email address). To these ends, the following validation functions are available:

- `str()` - Passes string values through, will ensure an value is present unless a `default` value is given. Note that an empty string is considered a valid value - if this is undesirable you can easily create your own validator (see below)
- `bool()` - Parses env var strings `"1"`, `"0"`, `"true"`, `"false"`, `"t"`, `"f"` into booleans
- `num()` - Parses an env var (eg. `"42"`, `"0.23"`, `"1e5"`) into a Number
- `email()` - Ensures an env var is an email address
- `host()` - Ensures an env var is either a domain name or an ip address (v4 or v6)
- `port()` - Ensures an env var is a TCP port (1-65535)
- `url()` - Ensures an env var is a url with a protocol and hostname
- `json()` - Parses an env var with `JSON.parse`

Each validation function accepts an (optional) object with the following attributes:

- `choices` - An Array that lists the admissible parsed values for the env var.

A página do envalid no npmjs tem uma documentação completa dos validadores disponíveis

express JS

Validando as variáveis de ambiente

- Agora que implementamos a função **getEnv**, podemos usá-la em nosso arquivo `index.ts`

```
import express, { type Request, type Response } from "express";

import { getEnv } from "../utils/getEnv.js";

const app = express();
const { PORT } = getEnv();

app.get("/", (req: Request, res: Response) => {
  res.send("Hello world!");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}.`);
});
```

Validando o validEnv

- Agora que definimos o `validEnv`, podemos usá-lo em nosso aplicativo. Vamos usar o `nodemon` para monitorar as alterações no código e reiniciar o aplicativo automaticamente.

```
david ~/dev/sn $ npm start
```

```
> sn@1.0.0 start
```

```
ar> nodemon --watch src --exec tsx src/index.ts
```

```
[nodemon] 3.1.14
[nodemon] to restart at any time, enter
[nodemon] watching path(s): src/**/*.
[nodemon] watching extensions: ts,js
[nodemon] starting `tsx src/index.ts`
```

```
=====
Missing environment variables:
NODE_ENV: undefined
```

```
=====
Exiting with error code 1
```

```
[nodemon] app crashed - waiting for file changes
before starting...
```

A aplicação passará a reportar um erro caso o arquivo `.env` não contenha todas as variáveis

Validando o ambiente de desenvolvimento

- Agora que definimos o `validEnv`, podemos usá-lo em nosso

```
npm start — npm start
~/dev/sn

david ~/dev/sn $ npm start

> sn@1.0.0 start
> nodemon --watch src --exec tsx src/index.ts

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): src/**/*.ts
[nodemon] watching extensions: ts,json
[nodemon] starting `tsx src/index.ts`
Server running on port 3333.

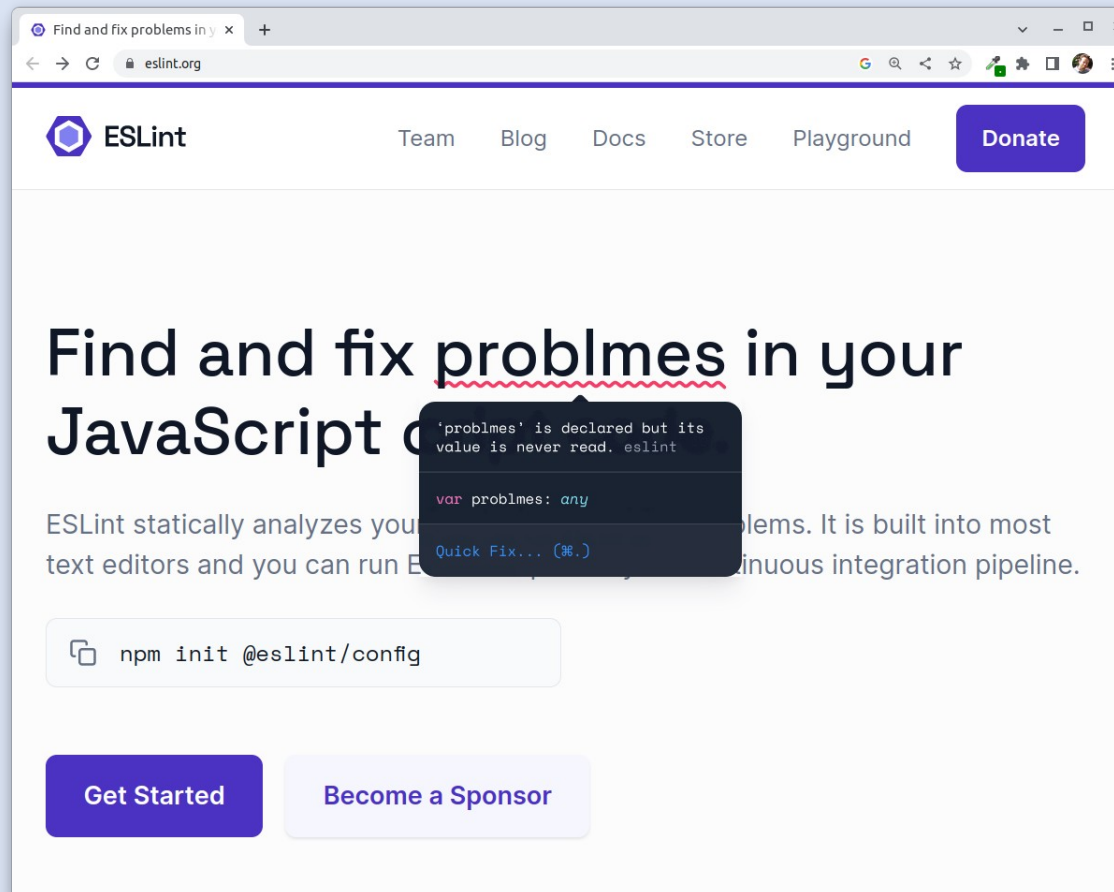
before starting...
```

O erro para de ocorrer quando atualizamos o `.env`



ESLint e Prettier

- **ESLint** é uma ferramenta de **lint**, que analisa o código em busca de erros de estilo e de sintaxe que possam levar a bugs



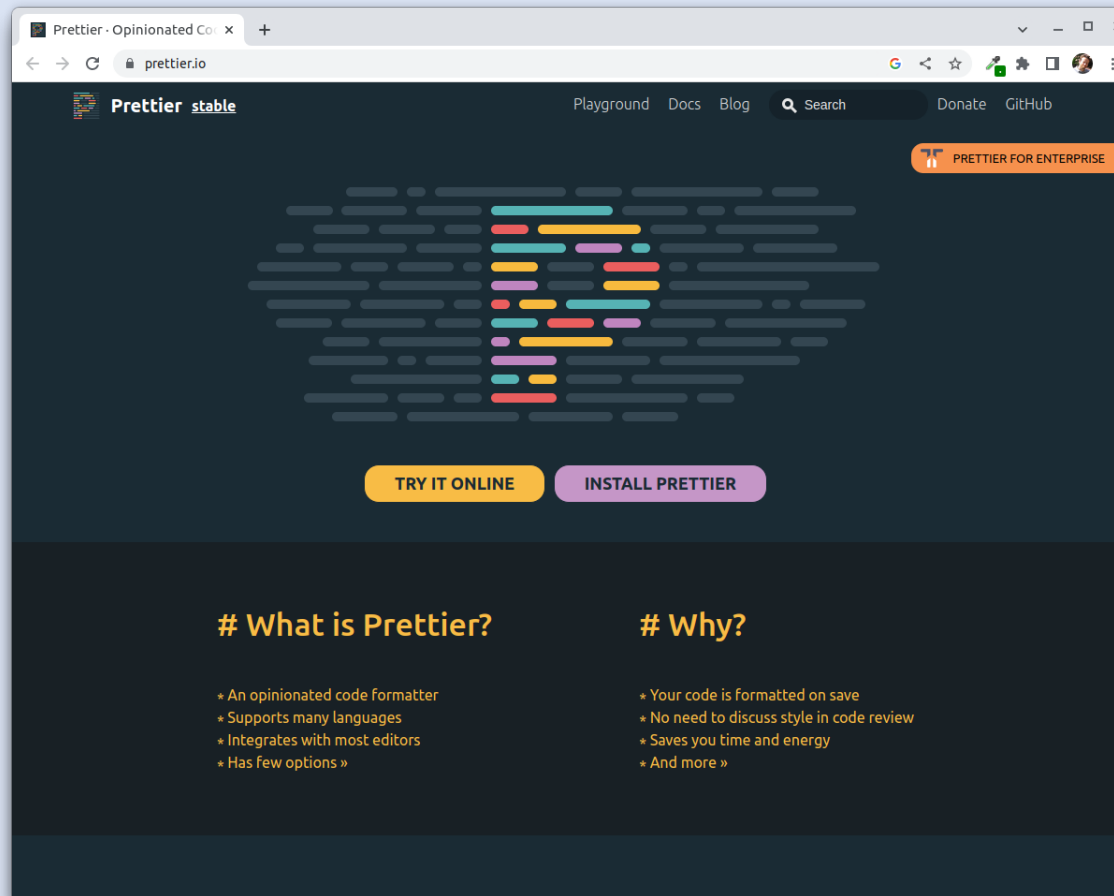
ESLint e Prettier

- **ESLint** é uma ferramenta de lint, que analisa o código em busca de bugs. A listagem a seguir mostra os **Linters** mais usados para as linguagens de programação mais populares

- JavaScript/TypeScript - **ESLint**
- Python - **Ruff** ou **Pylint**
- Java - **Checkstyle**
- C/C++ - **Clang-Tidy**
- C# - **Roslyn Analyzers**
- Go - **golangci-lint**
- Rust - **Clippy**
- PHP - **PHP_CodeSniffer**
- Ruby - **RuboCop**
- Kotlin - **Detekt**
- Swift - **SwiftLint**
- Bash - **ShellCheck**

ESLint e Prettier

- **Prettier** é uma ferramenta para **formatação** de código, que possibilita que os códigos sejam formatados automaticamente



ESLint e Prettier

- **Prettier** é uma ferramenta para formatação de código, que possibilita padronizar automaticamente o código.

A listagem a seguir mostra os **Formatadores** mais usados em diferentes linguagens de programação

- JavaScript/TypeScript - **Prettier**
- Python - **Black** ou **Ruff Formatter**
- Java - **google-java-format**
- C/C++ - **ClangFormat**
- C# - **dotnet format**
- Go - **gofmt**
- Rust - **rustfmt**
- PHP - **PHP CS Fixer**
- Ruby - **RuboCop**
- Kotlin - **ktlint**
- Swift - **SwiftFormat**
- Bash - **shfmt**

Adicionando o ESLint no projeto

- Para instalar o **ESLint** em seu projeto, basta adicionar o pacote eslint como uma dependência de desenvolvimento

```
$ npm i -D eslint
```

- Para configurar o ESLint, usamos o comando abaixo -

```
$ npm init @eslint/config@latest
```

```
david@coyote:~/dev/sn
```

```
david ~/dev/sn on ↗ main $ npm init @eslint/config@latest
```

```
> gc@1.0.0 npx
```

```
> "create-config"
```

```
@eslint/create-config: v2.0.0
```

✓ What do you want to lint? · javascript

✓ How would you like to use ESLint? · problems

✓ What type of modules does your project use? · [esm](#)

✓ Which framework does your project use? · none

✓ Does your project use TypeScript? · No / [Yes](#)

✓ Where does your code run? : `node`

✓ Which language do you want your configuration file be written in? · js

i The config that you've selected requires the following dependencies:

```
eslint, @eslint/js, globals, typescript-eslint
```

✓ Would you like to install them now? · No / [Yes](#)

✓ Which package manager do you want to use? · npm

☕ Installing...

```
added 19 packages, and audited 180 packages in 6s
```

60 packages are looking for funding

```
run `npm fund` for details
```

found 0 vulnerabilities

✓ Successfully created /home/david/dev/sn/eslint.config.js file.

express JS

Extension: ESLint



ESLint

Microsoft  microsoft.com

 52,379,126

★★★★☆ (250)

Integrates ESLint JavaScript into VS Code.

Disable

Uninstall



Auto Update



DETAILS

FEATURES

CHANGELOG

Installation

VS Code

Para uma completa compatibilidade do ESLint com o VS Code, recomenda-se instalar a extensão ESLint do editor



Azure Pipelines

succeeded

Version

3.0.34

Last

1 month ago

Updated

Size

898.20KB

Integrates [ESLint](#) into VS Code. If you are new to ESLint

check the [documentation](#)

Adicionando o ESLint no projeto

- Após a configuração do ESLint, será criado um arquivo **eslint.config.mjs** com o seguinte conteúdo:

```
import js from "@eslint/js";
import globals from "globals";
import tseslint from "typescript-eslint";
import { defineConfig } from "eslint/config";

export default defineConfig([
  {
    files: ["**/*.js,mjs,cjs,ts,mts,cts"],
    plugins: { js },
    extends: ["js/recommended"],
    languageOptions: { globals: globals.node }
  },
  tseslint.configs.recommended,
]);
```

Adicionando o ESLint no projeto

- Após a configuração, já criado um arquivo **eslint.config.mjs** com o seguinte conteúdo:

```
import js from "@eslint/js";
import globals from "eslint-config-eslint";
import tseslint from "typescript-eslint";
import { defineConfig } from "eslint/config";

export default defineConfig([
  {
    files: ["src/**/*..{js,mjs,cjs,ts,mts,cts}"],
    plugins: { js },
    extends: ["js/recommended"],
    languageOptions: { globals: globals.node },
  },
  tseslint.configs.recommended,
]);
```

No arquivo gerado, configure o ESLint para avaliar apenas arquivos do diretório **src**

A partir desse momento, o ESLint vai passar a acusar eventuais problemas no código

TS index.ts 1, M

eslint.config.js M

src > TS index.ts > ...

```
3 const app = express();
```

```
4 const PORT = 3333;
```

```
5 const novavariavel = 123; 'novavariavel' is assigned a value but never used.
```

'novavariavel' is declared but its value is never read. ts(6133)

'novavariavel' is assigned a value but never used. eslint([@typescript-eslint/no-unused-vars](#))

```
const novavariavel: 123
```

View Problem (Alt+F8)

Quick Fix... (Ctrl+.)

Fix (Ctrl+I)

Adicionando o ESLint no projeto

- Após a configuração do ESLint, será criado um arquivo **eslint.config.mjs** com o seguinte conteúdo:

```
import js from "@eslint/js";
import globals from "globals";
import tseslint from "typescript-eslint";
import { defineConfig } from "eslint/config";

export default defineConfig([
  {
    files: ["src/**/*..{js,mjs,cjs,ts,mts,cts}"],
    plugins: { js },
    extends: ["js/recommended"],
    languageOptions: { globals: globals.node },
  },
  tseslint.configs.recommended,
  { rules: { "@typescript-eslint/no-unused-vars": "off" } },
]);
```

Possíveis
valores:
**off, warn,
error**



sn

2



Update



TS index.ts M X

eslint.config.js M

src > TS index.ts > ...

You, 6 minutes ago | 1 author (You)

```
1 import express, { type Request, type Response } from "express";
2
3 const app = express();
4 const PORT = 3333;
5 const novavariavel = 123;
6
7 app.get("/", (req: Request, res: Response) => {
8   res.send("Hello world!");
9 });
10
11 app.listen(PORT, () => {
12   console.log(`Server running on port ${PORT}.`);
13 });
```

Com a regra desabilitada, o ESLint não acusa erros



main*



1 ↓ 1 ↑



Launchpad



0



0



Turbo v3.27.0



Prettier

express JS

Rules Reference - ESLint

eslint.org/docs/latest/rules/#suggestions

ESLint

TeamBlogDocsStorePlaygroundCode ExplorerDonate

Version

v10.9.0

Search

Q

Powered by  algolia

USE ESLINT IN YOUR PROJECT

Getting Started

Core Concepts

Glossary

Configure ESLint

Configuration File

Configure Language

Configure Rules

Rules Reference

Table of Contents

Possible Problems

Suggestions

Layout & Formatting

Deprecated

Removed

Rules in ESLint are grouped by type to help you understand their purpose. Each rule has emojis denoting:

✓

🔑



No burnout.

Fewer vulns. Pay maintainers like Pydantic Logfire does! [Join the Open Source Pledge.](#) 

opensourcepledge.com

Ads by EthicalAds

Em <https://eslint.org/docs/latest/rules/> é possível ver todas as rules do Eslint disponíveis para JavaScript

The screenshot shows a web browser with the URL `typescript-eslint.io/rules/`. The page has a navigation bar with links for `Overview`, `Rules`, `Docs`, and `Blog`. The `Rules` link is active. The main content area is titled `Overview` and contains the following text: `@typescript-eslint/eslint-plugin` includes over 100 rules that detect best practice violations, bugs, and/or stylistic issues specifically for TypeScript code. All of our rules are listed below. Below this text is a green box with a lightbulb icon and the word `TIP`, containing the text: `Instead of enabling rules one by one, we recommend using one of our pre-defined configs to enable a large set of recommended rules.` To the right of the main content is a sidebar with a search bar and a list of categories: `Rules`, `Filtering`, `Config Group (*)`, `Metadata`, `Extension Rules`, and `Frozen Rules`. On the left side of the page, there is a list of rule names, including `adjacent-overload-signatures`, `array-type`, `await-thenable`, `ban-ts-comment`, `ban-tslint-comment`, `class-literal-property-style`, `class-methods-use-this`, `consistent-generic-constructors`, `consistent-indexed-object-style`, `consistent-return`, `consistent-type-assertions`, `consistent-type-definitions`, `consistent-type-exports`, `consistent-type-imports`, and `default-param`. A blue box at the bottom of the page contains the text: `Em https://typescript-eslint.io/rules/ é possível ver todas as rules do Eslint disponíveis para TypeScript`.

Overview | typescript-esl x +

← → ↺ % typescript-eslint.io/rules/

typescript-eslint Docs Rules Blog v8.67.0 Playground

Overview

@typescript-eslint/eslint-plugin includes over 100 rules that detect best practice violations, bugs, and/or stylistic issues specifically for TypeScript code. All of our rules are listed below.

TIP

Instead of enabling rules one by one, we recommend using one of our pre-defined configs to enable a large set of recommended rules.

Rules

The rules are listed in alphabetical order. You can optionally filter them based on these categories:

- Rules
- Filtering
- Config Group (*)
- Metadata
- Extension Rules
- Frozen Rules

adjacent-overload-signatures
array-type
await-thenable
ban-ts-comment
ban-tslint-comment
class-literal-property-style
class-methods-use-this
consistent-generic-constructors
consistent-indexed-object-style
consistent-return
consistent-type-assertions
consistent-type-definitions
consistent-type-exports
consistent-type-imports
default-param

Em <https://typescript-eslint.io/rules/> é possível ver todas as rules do Eslint disponíveis para TypeScript

Adicionando o Prettier no projeto

- Para instalar o **Prettier** em seu projeto, basta adicionar o pacote prettier como uma dependência de desenvolvimento

```
$ npm i -D prettier
```

- Para configurarmos o Prettier, podemos criar um arquivo **.prettierrc** com as seguintes diretrizes de formatação

```
{  
  "printWidth": 80,  
  "tabWidth": 2,  
  "singleQuote": false,  
  "trailingComma": "all",  
  "semi": false,  
  "arrowParens": "always"  
}
```

Options · Prettier

prettier.io/docs/options.html

Prettier stable

Playground Docs Blog Donate

Search Ctrl K

About

- What is Prettier?
- Why Prettier?
- Prettier vs. Linters
- Option Philosophy
- Rationale

Usage

- Install
- Ignoring Code
- Integrating with Linters
- Pre-commit Hook
- Plugins
- CLI
- API
- Browser
- Run Prettier on CI

Configuring Prettier

- Options**
- Configuration
- Sharing config

Options

Prettier ships with a handful of format options.

To learn more about Prettier's stance on options – see the [Option Philosophy](#).

If you change any options, it's recommended to do it via a [configuration file](#). This way the Prettier CLI, [editor integrations](#) and other tooling knows what options you use.

Experimental Ternaries

Try prettier's [new ternary formatting](#) before it becomes the default behavior.

Valid options:

- `true` - Use curious ternaries, with the question mark after the condition.
- `false` - Retain the default behavior of ternaries; keep question marks on the same line as the consequent.

Default	CLI Override	API Override

Experimental Ternaries

- Experimental Operator Position
- Print Width
- Tab Width
- Tabs
- Semicolons
- Quotes
- Quote Props
- JSX Quotes
- Trailing Commas
- Bracket Spacing
- Object Wrap
- Bracket Line
- [Deprecated] JSX Brackets
- Arrow Function Parentheses
- Range
- Parser
- File Path

Em <https://prettier.io/docs/en/options.html> é apresentado a lista de opções do pacote Prettier

Prettier - Code formatter

Prettier  prettier.io

★★★★☆ (492)

71,023,009

Sponsored

Code formatter using prettier

Disable

Uninstall



Podemos usar a extensão Prettier do VSCode para formatar o código automaticamente a cada salvamento

Prettier Formatter for Visual Studio Code

Prettier is an opinionated code formatter. It enforces a consistent style by parsing your code and re-printing it with the same rules that take the prettier. It's built on top of the [Prettier](#) code formatter.

vscode

Version 12.4.0

Last Updated 5 months ago

Size 14.01MB



sn



2



TS index.ts 2, M

Extension: Prettier - Code formatter

eslint



main*



1 ↓ 1 ↑



2



0



Launchpad



2



0



Turbo v3.27.0



Go Live



Go Live



Go Live



Go Live



Go Live



Go Live



Go Live

Adicionando o Prettier no projeto

- E agora podemos adicionar os scripts **lint**, **lint:fix** e **format** no **package.json** para rodar o eslint e o prettier

```
"scripts": {  
  "start": "tsx watch src/index.ts",  
  "build": "tsc",  
  "typecheck": "tsc --noEmit",  
  "start:prod": "node dist/index.js",  
  "lint": "eslint src/",  
  "lint:fix": "eslint --fix src/",  
  "format": "prettier --write src/"  
},
```

Roteamento

- Roteamento é usado para definir quais middlewares serão usados para responder quais tipos de URLs

```
app.get("/", (req, res) => {  
  res.send("Bem-vindo ao meu site!");  
});
```

Executado só
na requisição
GET /

```
app.get("/about", (req, res) => {  
  res.send("Bem-vindo à página sobre!");  
});
```

Executado só
na requisição
GET /sobre

```
app.use((req, res) => {  
  res.statusCode = 404;  
  res.send("404!");  
});
```

Executado
se a rota não
for **GET /** nem
GET /sobre

Roteamento

■ F... U... es serão

Bem-vindo ao meu site!

localhost:3333/sobre

localhost:3333/sobre

Bem-vindo à página sobre!

localhost:3333/oii

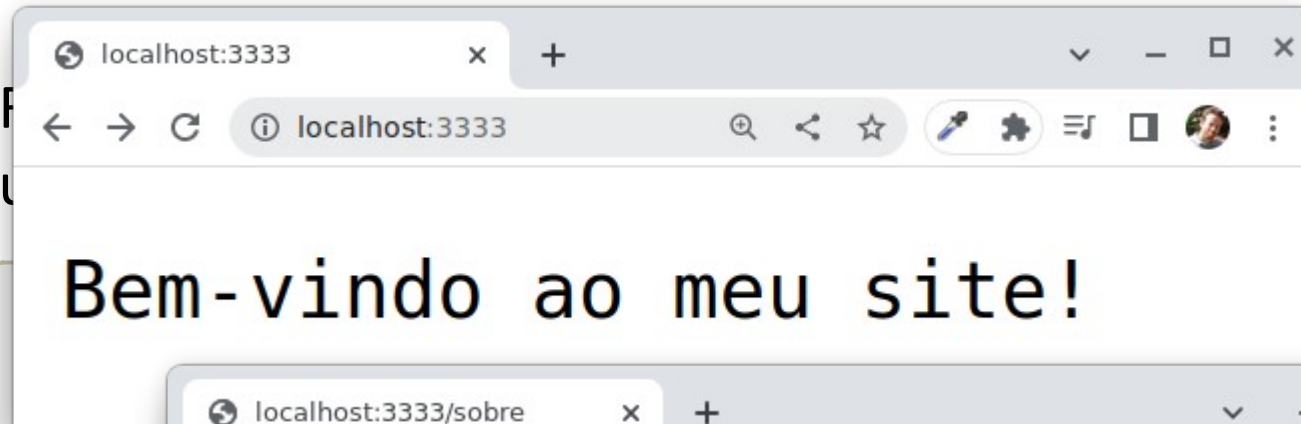
localhost:3333/oii

404!

```
app.get(  
  res.  
});
```

```
app.use(  
  res.status(  
  res.send(  
});
```

Roteamento



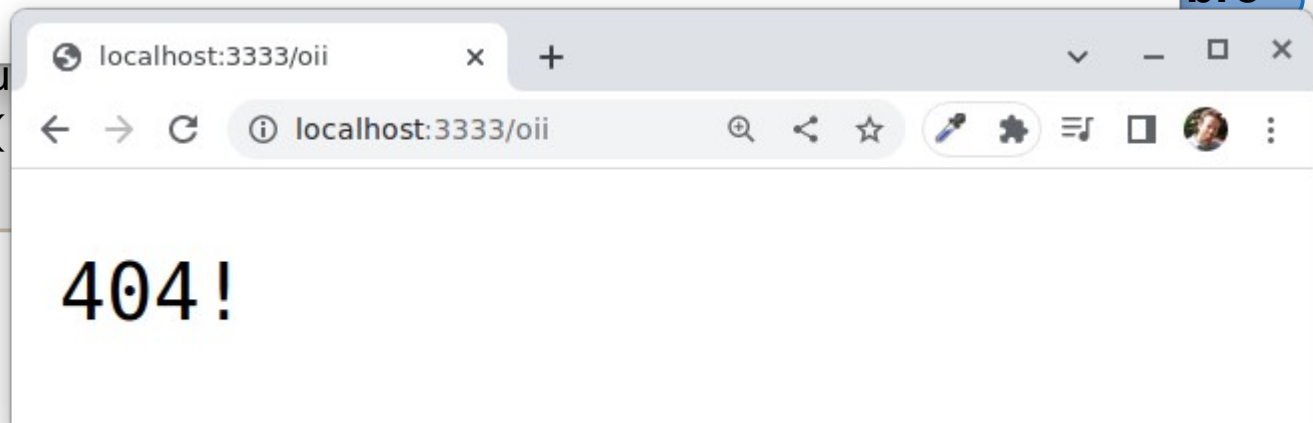
es serão

do só

Além de **app.get()**, também existe o método **app.post()**, que é usado para responder às requisições do tipo POST

bre

```
app.use  
res.statu  
res.send(  
});
```



Roteamento

- Além dos nomes de rotas fixos, o Express aceita o uso de rotas mais complexas, incluindo **expressões regulares**

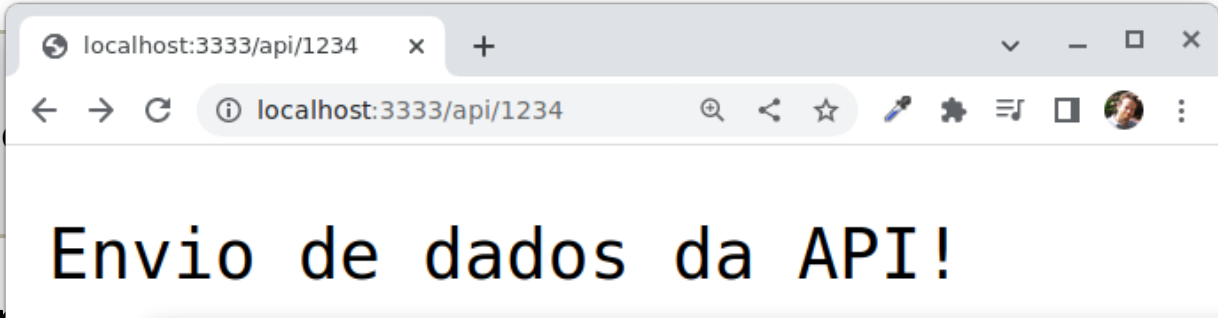
```
app.get( /^\/(api|rest)\/.+$/, (req, res) => {  
  res.send("Envio de dados da API!");  
});
```

- Outra possibilidade é a **passagem de parâmetros** através da URL, tal como demonstrado no exemplo a seguir

```
app.get("/bemvindo/:nome", (req, res) => {  
  res.send(`Seja bem vindo ${req.params.nome}`);  
});
```

Roteamento

- Além dos nomes de rotas fixos, o Express aceita o uso de rotas mais complexas, incluindo **expressões regulares**



localhost:3333/api/1234

Envio de dados da API!

This screenshot shows a web browser window with the address bar set to 'localhost:3333/api/1234'. The page content displays 'Envio de dados da API!' in a monospace font. The browser's developer tools are open on the left, showing a snippet of JavaScript code: `app.r
});`.

- Outra forma de definir rotas é através da URL, tal como



localhost:3333/bemvindo/david

Seja bem vindo david

This screenshot shows a web browser window with the address bar set to 'localhost:3333/bemvindo/david'. The page content displays 'Seja bem vindo david' in a monospace font. The browser's developer tools are open on the left, showing a snippet of JavaScript code: `app.get(
res.ser
});`.

Arquivo Separado de Rotas

- Ao invés de definir todas as rotas no arquivo principal da aplicação, é possível criar um arquivo separado para elas
- Para isso, podemos usar a classe **express.Router**, que retorna um middleware com um sistema de roteamento completo

```
// Módulo de rotas, arquivo /router/router.ts  
  
import { Router } from 'express';  
const router = Router();  
  
router.get('/', (req, res) => {  
  res.send('Página principal do site');  
});  
  
router.get('/sobre', (req, res) => {  
  res.send('Página sobre');  
});  
  
export default router;
```

Arquivo Separado de Rotas

- Em seguida, podemos carregar o módulo de roteamento no arquivo principal da app

```
// Módulo principal, arquivo /src/index.ts  
  
import express from 'express';  
import router from './router/router';  
  
const app = express();  
app.use(router);  
  
app.listen(3000, () => {  
  console.log("Express app iniciada na porta 3000.");  
});
```

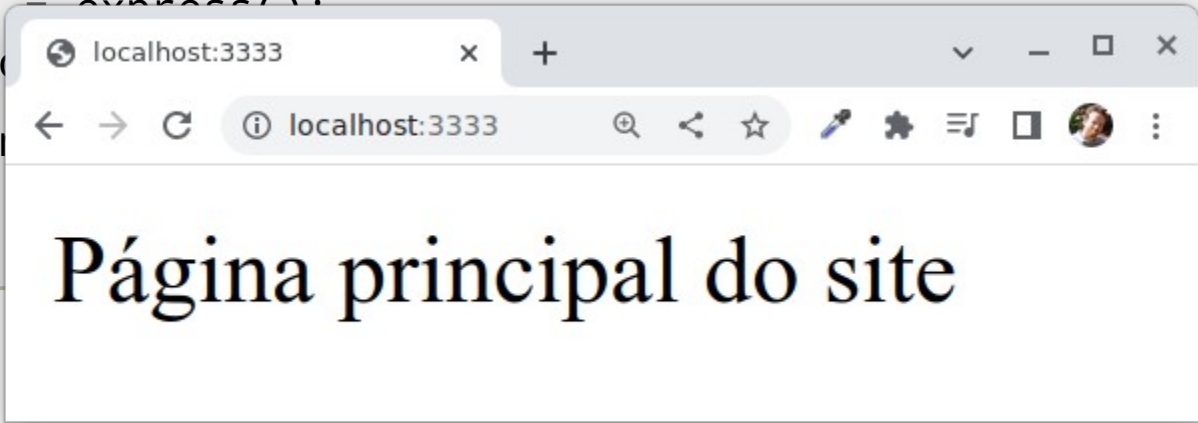
Arquivo Separado de Rotas

- Em seguida, podemos carregar o módulo de roteamento no arquivo principal da app

```
// Módulo principal, arquivo /src/index.ts
```

```
import express from 'express';  
import router from './router/router';
```

```
const app = express();  
app.use(router);  
app.listen(3333, () => {  
  console.log('Servidor rodando na porta 3333');  
});
```



Página principal do site

Arquivo Separado de Rotas

- Em seguida, podemos carregar o módulo de roteamento no arquivo principal da app

```
// Módulo principal, arquivo /src/index.ts  
  
import express from 'express';  
import router from './router/router';
```

O **Router** surgiu na versão 4.0 do **Express**, e é uma alternativa ao método tradicional de criação das rotas. A função do Router é permitir a definição das rotas em um módulo separado.

```
console.log('...');  
});
```

Página principal do site

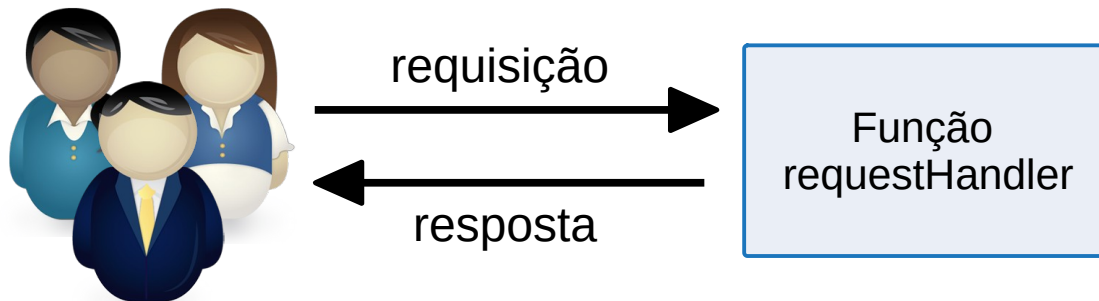
Os Fundamentos do Express

- O framework express provê 4 funcionalidades principais, que serão vistas ao longo dos próximos slides:
 - **Middleware** – São um conjunto de funções que tem acesso ao objeto de solicitação (req) e ao o objeto de resposta (res)
 - **Routing** – As rotas definem como o aplicativo irá responder às solicitações (URI + HTTP Method) dos clientes
 - **Extensões aos objetos request e response** – Express estende os objetos request e response com novos métodos e propriedades
 - **Views** – As views permitem criar conteúdo HTML de forma dinâmica

Middleware

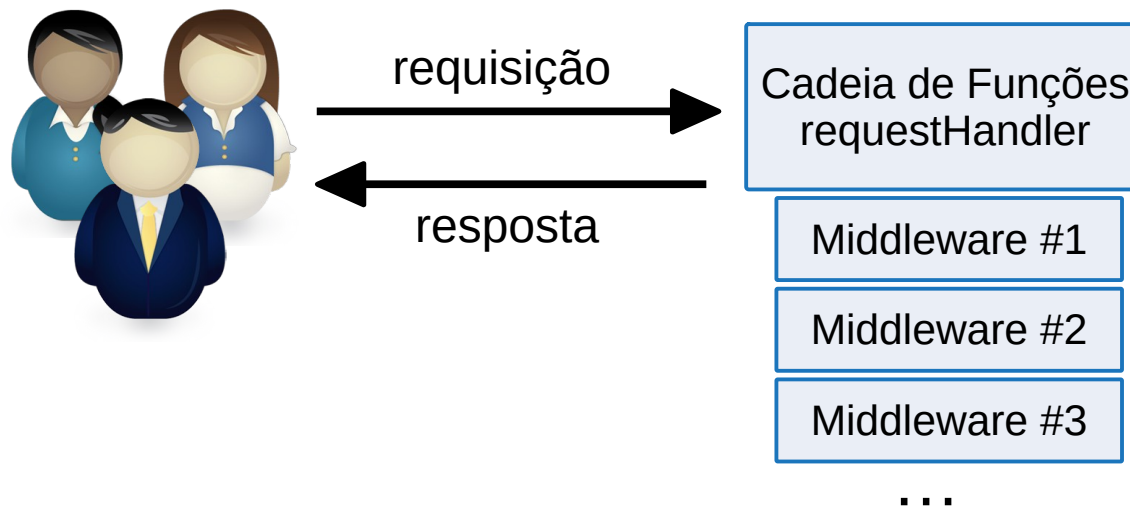
- Usando Node.js puro, usamos o callback de **http.createServer** para responder todas as requisições dos usuários

```
http.createServer((req, res) => {  
  console.log(`Requisição do usuário: ${req.url}`);  
  res.end("Instituto de Computação!");  
});
```



Middleware

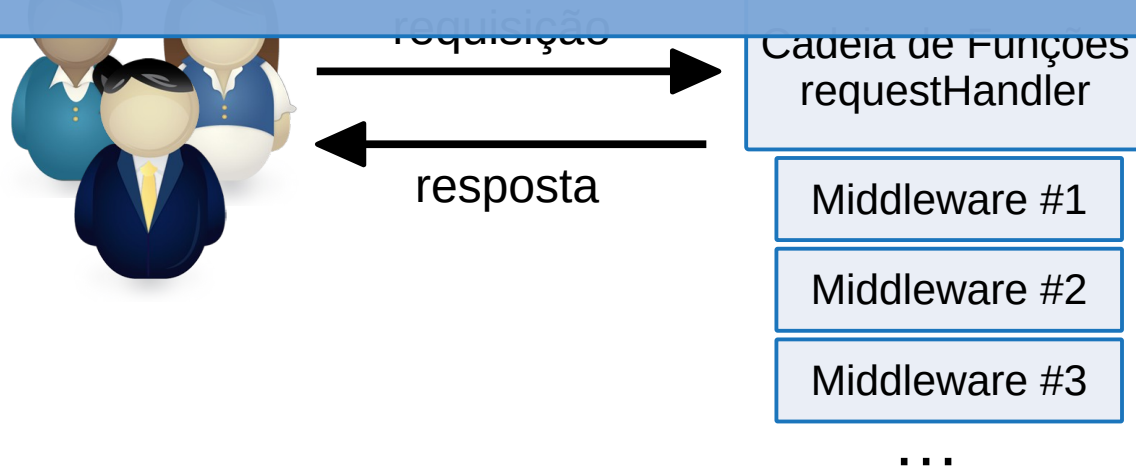
- Os **middlewares** são funções similares ao callback de **http.createServer**, mas possuem uma diferença fundamental: ao invés de um callback, podemos criar vários em sequência



Middleware

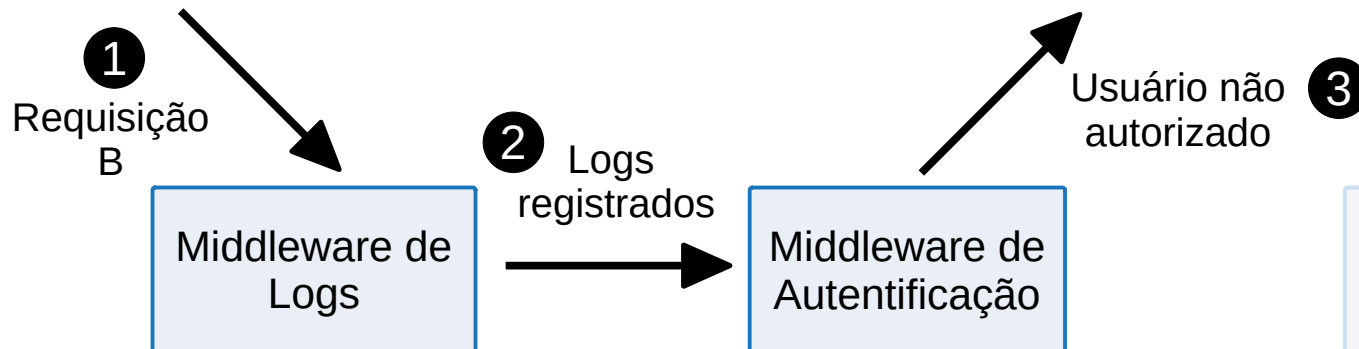
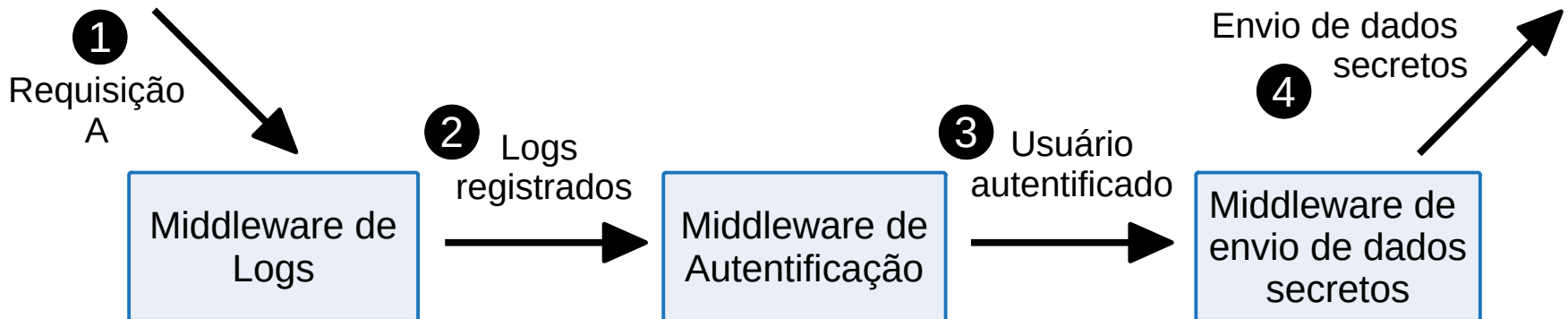
- Os **middlewares** são funções similares ao callback de **http.createServer**, mas possuem uma diferença fundamental: ao invés de um callback, podemos criar vários em sequência

Os middlewares não são uma invenção do Express, e estão presentes em vários outros frameworks: **Django, Laravel, Ruby on Rails**, etc.



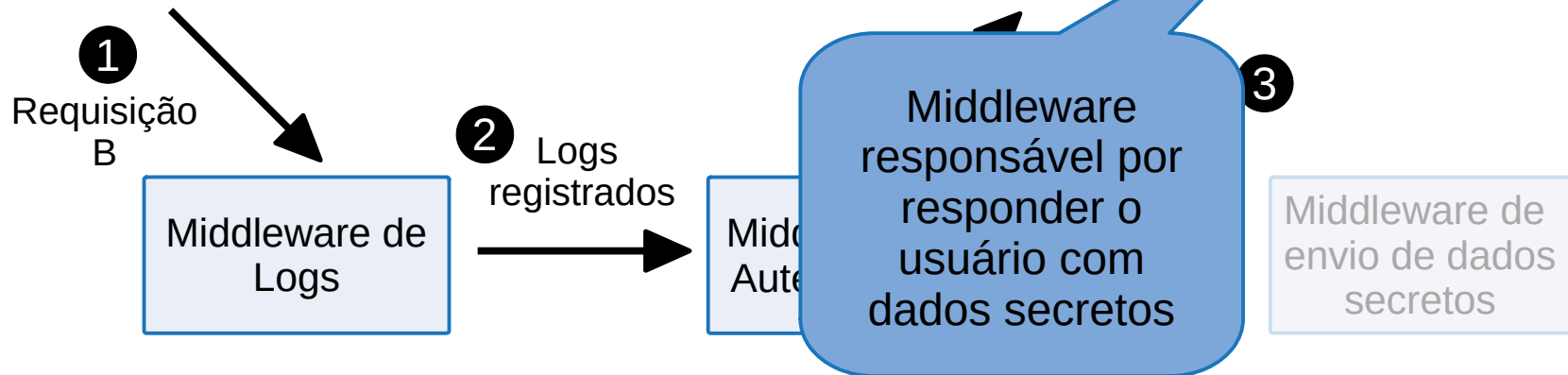
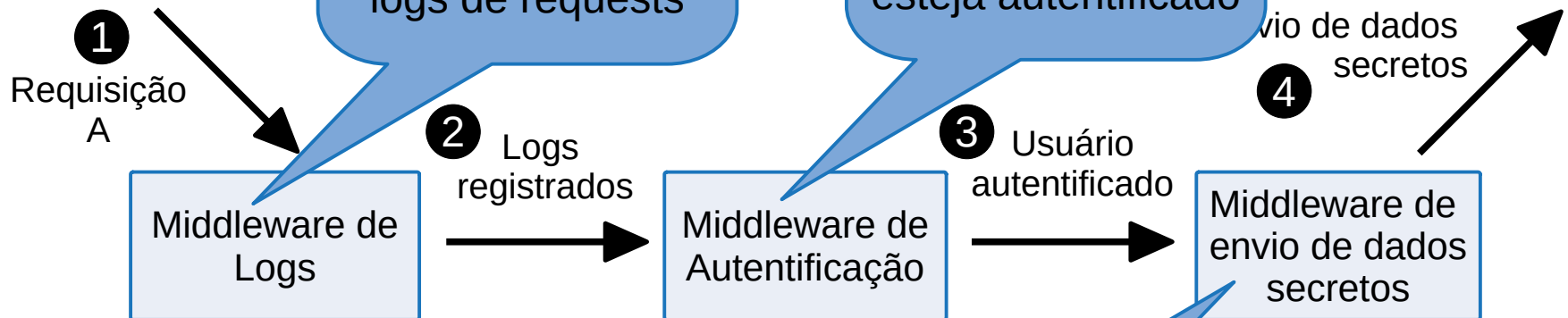
Middleware

- A Figura abaixo mostra um exemplo de app que autentica os usuários e retorna dados sigilosos para usuários autorizados



Middleware

- A Figura 2 mostra um exemplo de middleware de Logs e um exemplo de middleware de autenticação. O middleware de Logs é uma função responsável por criar logs de requests. O middleware de autenticação irá deixar o usuário seguir caso ele esteja autenticado.



Middleware

- A Figura 2 mostra um exemplo de middleware de Logs e um exemplo de middleware de autenticação. O middleware de Logs é uma função responsável por criar logs de requests. O middleware de autenticação irá deixar o usuário seguir caso ele esteja autenticado.

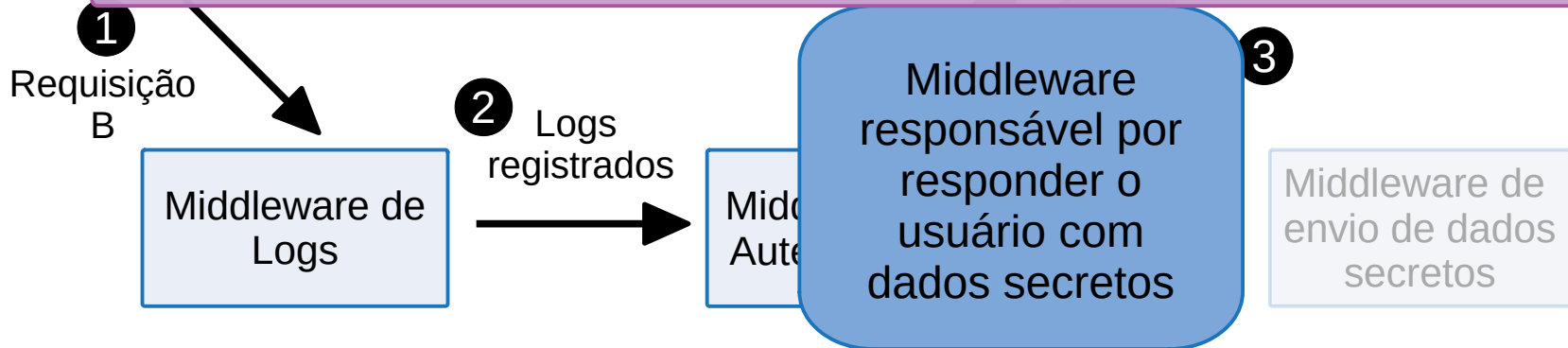
Neste exemplo, o middleware de Logs é uma função responsável por criar logs de requests

O middleware de autenticação irá deixar o usuário seguir caso ele esteja autenticado



Os middlewares são muito úteis porque permitem dividir a lógica da aplicação em várias partes menores

Além disso, muitos módulos de terceiros são projetados para se encaixarem perfeitamente no formato dos middlewares



Middleware

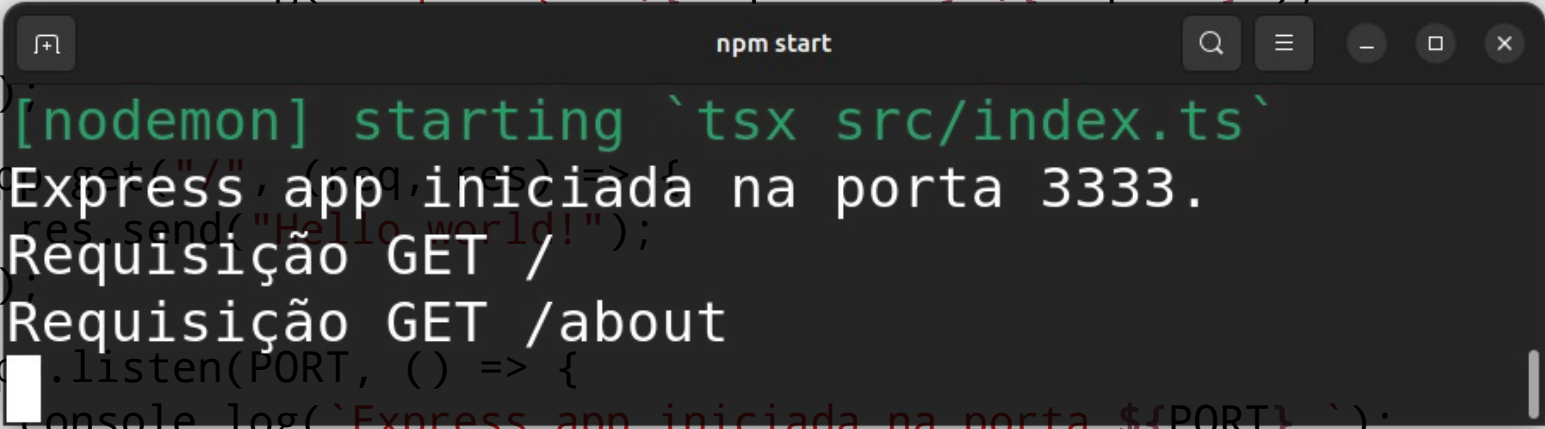
- Além dos parâmetros **Request (req)** e **Response (res)**, os middlewares também recebem o parâmetro **next**, que pode ser usado para chamar o próximo middleware da cadeia

```
app.use((req: Request, res: Response, next: NextFunction) => {  
  console.log(`Requisição ${req.method} ${req.url}`);  
  next();  
});  
  
app.get("/", (req, res) => {  
  res.send("Hello world!");  
});  
  
app.listen(PORT, () => {  
  console.log(`Express app iniciada na porta ${PORT}.`);  
});
```

Middleware

- Além dos parâmetros **Request (req)** e **Response (res)**, os middlewares também recebem o parâmetro **next**, que pode ser usado para chamar o próximo middleware da cadeia

```
app.use((req: Request, res: Response, next: NextFunction) => {  
  console.log(`Requisição ${req.method} ${req.url}`);  
});  
app.get("/", (req, res) => {  
  res.send("Hello world!");  
});  
app.listen(PORT, () => {  
  console.log(`Express app iniciada na porta ${PORT}.`);  
});
```



Middleware

- Dentro de cada middleware, também é possível alterar propriedades dos objetos **request (req)** e **response (res)**

```
app.use((req, res, next) => {  
  console.log(`Requisição ${req.method} ${req.url}`);  
  next();  
});  
  
app.use((req, res, next) => {  
  if (user.checkAuth(req)) {  
    next();  
  } else {  
    res.statusCode = 403;  
    res.json({ msg: "Usuário não autenticado" });  
  }  
});  
  
app.get("/") (req, res) => {  
  res.json({ dados_secretos: { codigo: 156234 } });  
});
```

Código HTTP
para
Forbidden

Segurança da API

- Ao invés de programar todos os middlewares de sua aplicação, podemos usar middlewares disponíveis no repositório do NPM
- Existem milhares de middlewares disponíveis para uso, e um dos mais conhecidos é o **Helmet**
 - O Helmet adiciona cabeçalhos HTTP de segurança que ajudam a proteger aplicações Express contra vulnerabilidades comuns
 - Ele garante que boas práticas de segurança HTTP estejam ativas por padrão na aplicação
- O primeiro passo para usar o Helmet é instalá-lo

```
$ npm i helmet
```

Segurança da API

- Ao invés de programar todos os middlewares de sua aplicação, podemos usar middlewares disponíveis no repositório do NPM
- Existem milhares de middlewares disponíveis para uso, e um dos mais populares é o Express.js

– O Helmet
protege

– Ele garante
por padrão

- O primeiro

```
import express from 'express';
import helmet from 'helmet';
import dotenv from 'dotenv';

dotenv.config();
const PORT = process.env.PORT ?? 3333;

const app = express();

app.use(helmet());
```

ajudam a
comuns
am ativas

retorna uma
função
middleware

Registro das Requisições

- Outro exemplo muito conhecido de moddleware é o **Morgan**
 - Ele é usado para registrar informações sobre as requisições HTTP, como método, URL, status da resposta e tempo de processamento
 - Também é muito útil para depuração e análise do tempo de resposta da aplicação, principalmente durante o desenvolvimento
- A instalação do Morgan é feita com os comandos abaixo:

```
$ npm i morgan  
$ npm i -D @types/morgan
```

Registro das Requisições

- Outro exemplo muito conhecido de middleware é o **Morgan**
 - Ele é usado para registrar informações sobre as requisições HTTP, como método, URL, status da resposta e tempo de processamento
 - Também é muito útil para depuração e análise do tempo de resposta da aplicação, principalmente durante o desenvolvimento

- A instalação é feita da seguinte maneira: abaixo:

```
$ npm i  
$ npm i
```

```
import express from 'express';  
import morgan from 'morgan';  
import dotenv from 'dotenv';  
  
dotenv.config();  
const PORT = process.env.PORT ?? 3333;  
  
const app = express();  
  
app.use(morgan('combined'));
```

Registro das Requisições

- Outro exemplo muito conhecido de moddleware é o **Morgan**
 - Ele é usado para registrar informações sobre as requisições HTTP, como método, URL, status da resposta e tempo de processamento
 - Também é muito útil para depuração e análise do tempo de resposta para cada requisição durante o desenvolvimento
- A instalação do Express com Morgan é feita da seguinte maneira:

```
david ~/dev/sn $ npm start
> sn@1.0.0 start
> nodemon --watch src --exec tsx src/index.ts
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): src/**/*.ts
[nodemon] watching extensions: ts,json
[nodemon] starting `tsx src/index.ts`
Server running on port 3333.
$ curl -i http://localhost:3333/
::1 - - [26/Jul/2026:09:17:37 +0000] "GET / HTTP/1.1" 304
$ curl -i http://localhost:3333/
::1 - - [26/Jul/2026:09:17:40 +0000] "GET / HTTP/1.1" 304
- "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"
::1 - - [26/Jul/2026:09:17:40 +0000] "GET / HTTP/1.1" 304
- "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36"

app.use(morgan('combined'));
```

Outras Bibliotecas Middleware

- Existem várias outras bibliotecas middleware muito usadas, dentre as quais podemos destacar:
 - **Compression** – comprime os dados enviados para o cliente (browser) usando gzip
 - **Express.json** – converte a propriedade body da requisição do usuário (req.body) para vários formatos, incluindo JSON
 - **Express-rate-limit** – limita a quantidade de requisições feitas por um cliente em determinado período
 - **CORS** – controla quais origens podem acessar os recursos da aplicação
 - **Multer** – processa uploads de arquivos enviados por formulários

Extensões de Req/Res

- **Request** (req) e **response** (res) são objetos passados para todas as funções que tratam as requisições de usuários
- Eles já estavam existiam no Node.js puro, mas o Express adicionou novas funcionalidades para esses dois objetos
- Por exemplo, o método **redirect()** foi adicionado pelo Express

```
res.redirect("/hello/world");  
res.redirect("http://expressjs.com");
```

Redireciona
o usuário para
a nova URL

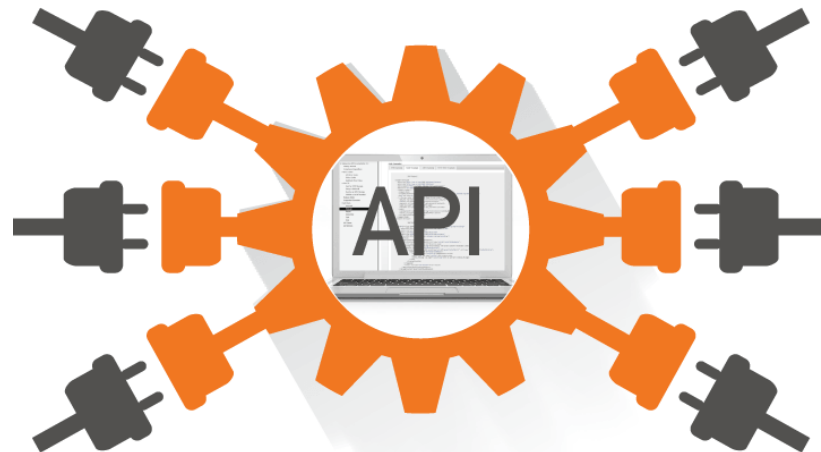
- Outro exemplo é o método **sendFile()**

```
res.sendFile("/path/to/song.mp3");
```

Envia o
arquivo para
o browser

REST APIs

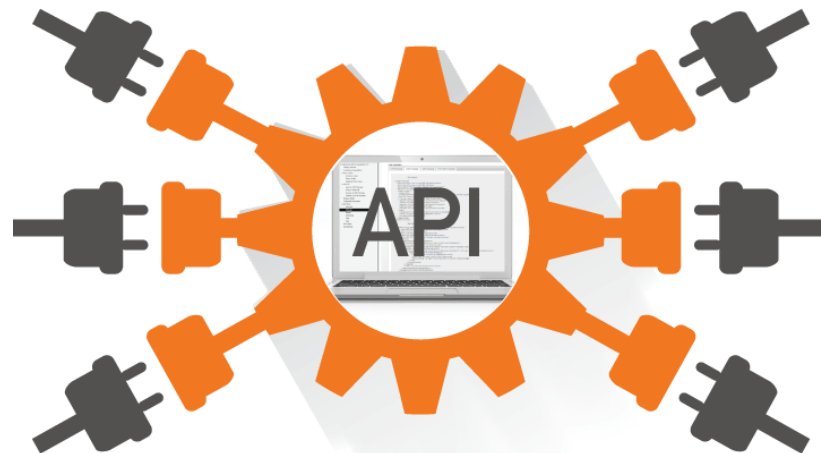
- O REST – Representational State Transfer – é caracterizado como um paradigma de desenvolvimento de software semelhante aos webservice
 - Nesse paradigma, um serviço (normalmente chamado de API) é fornecido para acesso e manipulação dos dados de uma aplicação
- API – Application Programming Interface – é um conjunto de rotinas usadas na comunicação entre duas partes de uma aplicação



REST APIs

- O REST – Representational State Transfer – é caracterizado como um paradigma de desenvolvimento de software semelhante aos webservice
 - Nesse paradigma, um serviço (normalmente chamado de API) é fornecido para acesso e manipulação dos dados de uma aplicação
- O Front, que consome os dados da API, pode ser desenvolvido através de vários tipos de frameworks frontend, como o **React**, **Angular** e o **Vue.js**

aplicação



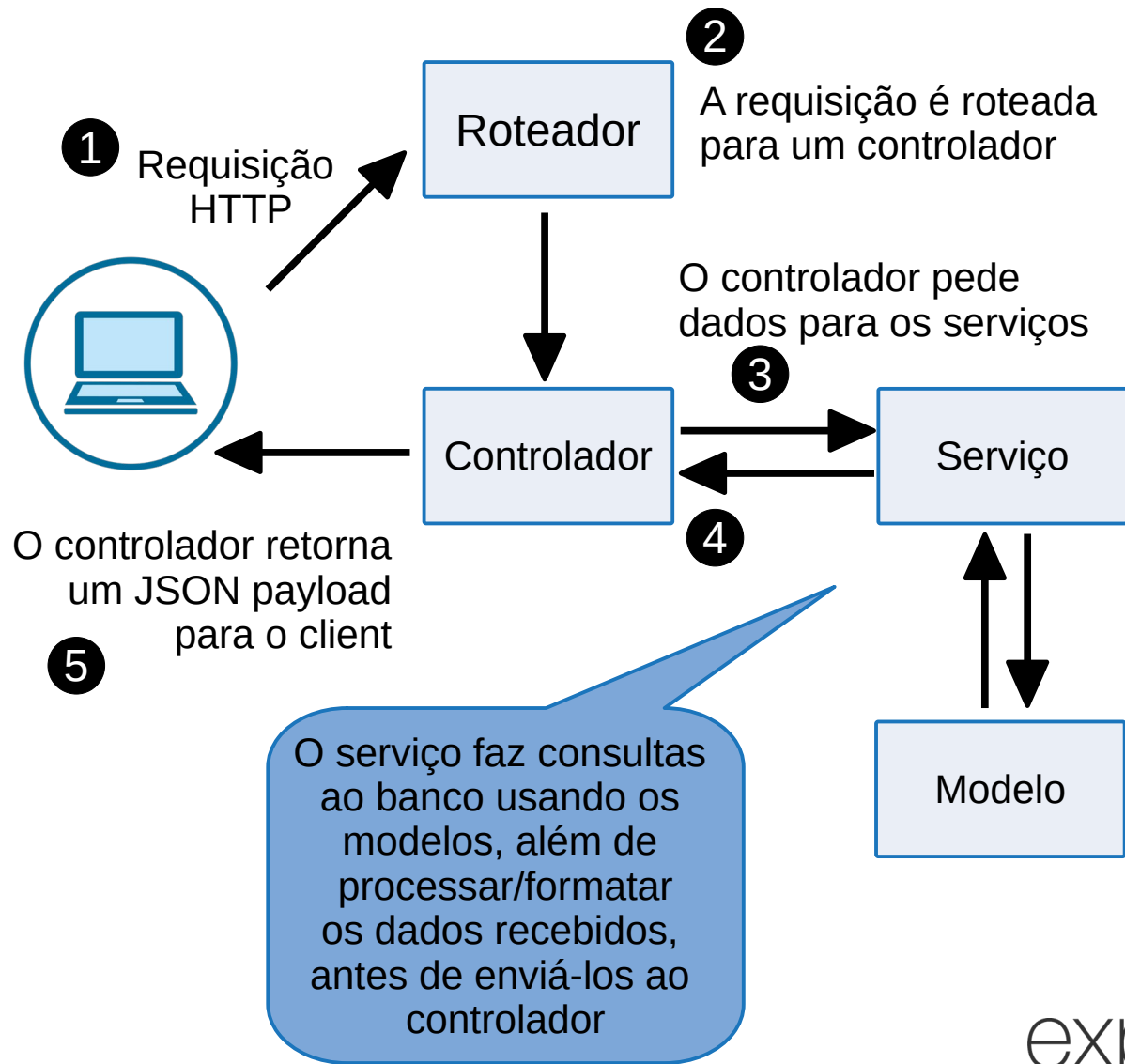
Sistema SmartNotes

- Como prática de desenvolvimento nesta etapa do curso, será desenvolvido uma API para um SPA de anotações pessoais, chamado **SmartNotes**



SmartNotes

Movendo para o padrão REST



Movendo para o padrão REST

- Para o padrão REST, optamos por organizar os arquivos de nossa aplicação usando o esquema abaixo
- O diretório **src** terá todos os arquivos fontes da aplicação, exceto os **modelos**

```
david@coyote:~/dev/pw/src $ tree -d
.
├── middlewares
├── resources
│   ├── note
│   └── user
└── router

6 directories
david@coyote:~/dev/pw/src $
```

Movendo para o padrão REST

- Para o padrão REST, optamos por organizar nossa aplicação usando o esquema abaixo
- O diretório `resources` terá os arquivos fonte, exceto os arquivos de configuração

O diretório **resources** terá um subdiretório para cada entidade da aplicação

Dentro dos subdiretórios haverá um roteador, um controlador, um serviço, um schema e um arquivo de tipos

```

david@coyote:~/dev/pw/src $ tree -d
.
├── middleware
└── resources

david@coyote:~/dev/pw/src/resources/note $ ls
note.controller.ts  note.schema.ts    note.types.ts
note.router.ts      note.service.ts

david@coyote:~/dev/pw/src/resources/note $
david@coyote:~/dev/pw/src $
```

Elementos de uma Requisição

- O **endpoint** é o caminho usado para fazer uma requisição, possuindo um **resource** e opcionalmente uma **query string**

<http://api.smartnotes.com/notes/?search=books>
resource ou path query string

- O **método HTTP** define o tipo de ação desejada pela requisição, sendo que os métodos mais usados são:
 - **Get**, usado para buscar dados do servidor
 - **Post**, usado para enviar dados para o servidor
 - **Put** e **Patch**, usado para atualizar dados
 - **Delete**, usado para apagar registros no servidor

Elementos de uma Requisição

- O **body** é o corpo da mensagem enviada na requisição, e é usado apenas com os métodos POST, PUT e PATCH
- Os **HTTP status codes** servem para o servidor notificar o cliente sobre qual o resultado da requisição HTTP
- Os principais códigos utilizados para as respostas de um endpoint são o 200 (OK), o 201 (CREATED), o 204 (NO CONTENT), o 404 (NOT FOUND) e o 400 (BAD REQUEST).



HTTP Cats

http.cat

HTTP Cats

Usage:

Descrição dos vários HTTP Status Code: <https://http.cat/>

`https://http.cat/[status_code]`

Note: If you need an extension at the end of the URL just add `.jpg`.

 100 Continue	 101 Switching Protocols	 102 Processing
		

O Controlador

- No controlador nós definimos as **actions** (funções) da API relacionadas com o resource **note**

```
// Arquivo src/resources/note/note.controller.ts  
  
import { Request, Response } from "express";  
  
const index = (req: Request, res: Response) => {};  
const create = (req: Request, res: Response) => {};  
const read = (req: Request, res: Response) => {};  
const update = (req: Request, res: Response) => {};  
const remove = (req: Request, res: Response) => {};  
  
export default { index, create, read, update, remove };
```

O Roteador

- O **Roteador** de cada resource contém as rotas associadas ao resource, referenciando as actions do controlador

```
// Arquivo src/resources/note/note.router.ts  
  
import { Router } from 'express';  
import noteController from './note.controller';  
const router = Router();  
  
// Note controller  
router.get('/', noteController.index);  
router.post('/', noteController.create);  
router.get('/:id', noteController.read);  
router.put('/:id', noteController.update);  
router.delete('/:id', noteController.remove);  
  
export default router;
```

O Roteador

- O **Roteador** de cada resource contém as rotas associadas ao resource, referenciando as actions do controlador

```
// Arquivo src/resources/note/note.router.ts
```

```
import { Router } from 'express';  
import noteController from './note.controller';  
const router = Router();
```

Embora não faça parte do CRUD, o objetivo da rota **/note** é listar as notas do usuário logado

```
router.post('/', noteController.create);  
router.get('/:id', noteController.read);  
router.put('/:id', noteController.update);  
router.delete('/:id', noteController.remove);  
  
export default router;
```

O Roteador

- O **Roteador** de cada resource contém as rotas associadas ao resource, referenciando as actions do controlador

```
// Arquivo src/resources/note/note.router.ts
```

```
import { Router } from 'express';  
import noteController from './note.controller';  
const router = Router();
```

```
// Embora não faça parte do CRUD, o objetivo da rota /note
```

Note que as rotas para **read**, **update** e **remove** terminam com a string **:id**, que representa um parâmetro utilizado para informar que nota se deseja **ler**, **atualizar** ou **apagar**

```
router.put('/:id', noteController.update);  
router.delete('/:id', noteController.remove);  
  
export default router;
```

O Roteador

- O **Roteador** de cada resource contém as rotas associadas ao resource, referenciando as actions do controlador

```
// Arquivo src/resources/note/note.router.ts
```

```
import { Router } from 'express';
```

```
// Arquivo src/router/v1Router.ts
```

```
import express from "express";
```

```
import noteRouter from "../resources/note/note.router";
```

```
const router = express.Router();
```

```
router.use("/note", noteRouter);
```

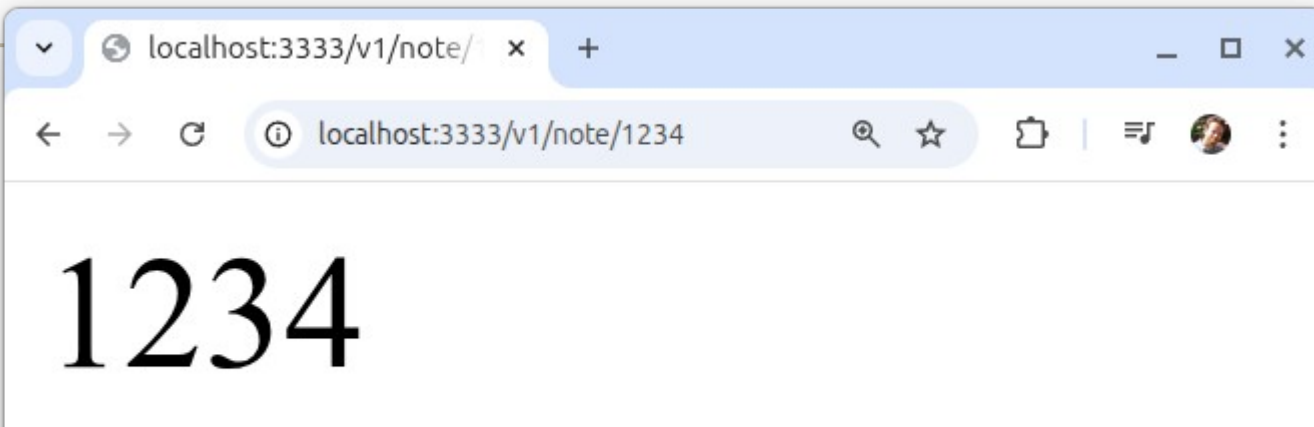
```
export default router;
```

```
export default router;
```

O Roteador

- Os **parâmetros** permitem passar dados informações adicionais para o endpoint desejado
 - Por exemplo, na url **http://localhost:3333/v1/note/1234**, o valor do parâmetro **id** é 1234
- Para ler o valor de **id** dentro de uma função, podemos usar o atributo **param** de **req** (objeto da requisição do usuário):

```
const read = (req: Request, res: Response) => {  
  const noteId = req.params.id;  
  res.send(noteId);  
};
```



O Roteador

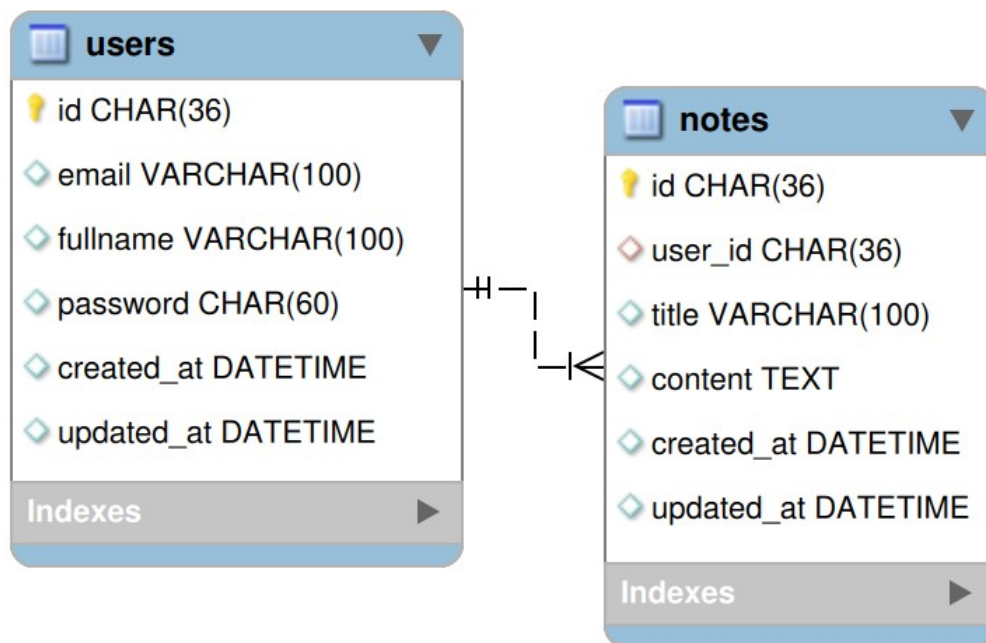
- O Express possui um middleware chamado **json()**, que é usado para extrair os dados do corpo da requisição (**req.body**)
- Para usá-lo, basta inserir a linha abaixo no arquivo **src/index.ts** antes da chamada ao middleware router:

```
// Arquivo src/index.ts  
...  
app.use(express.json());  
app.use(router);
```

- Após isso, o **express.json()** irá extrair os dados do **request body** das requisições e copiá-los no objeto **req.body**

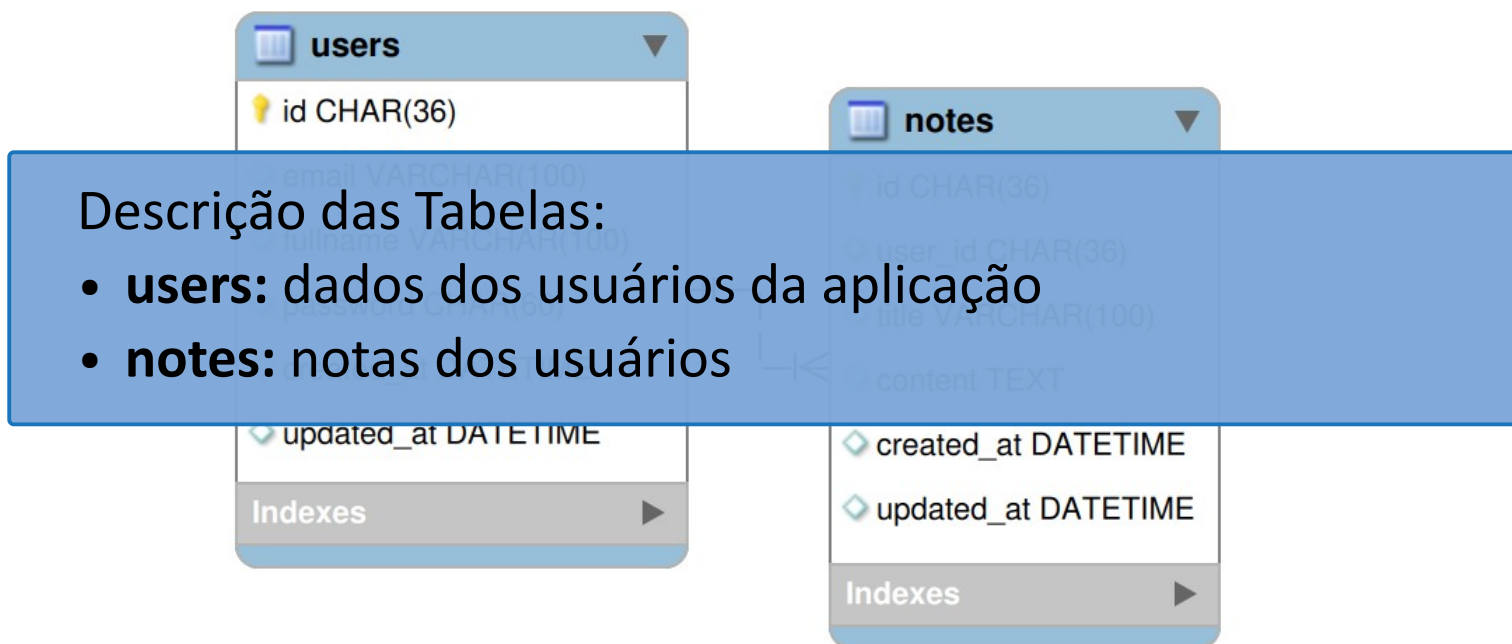
Esquema de banco de dados e ORM

- Nossa aplicação usará o ORM Prisma, e os modelos serão definidos no arquivo **prisma/schema.prisma**



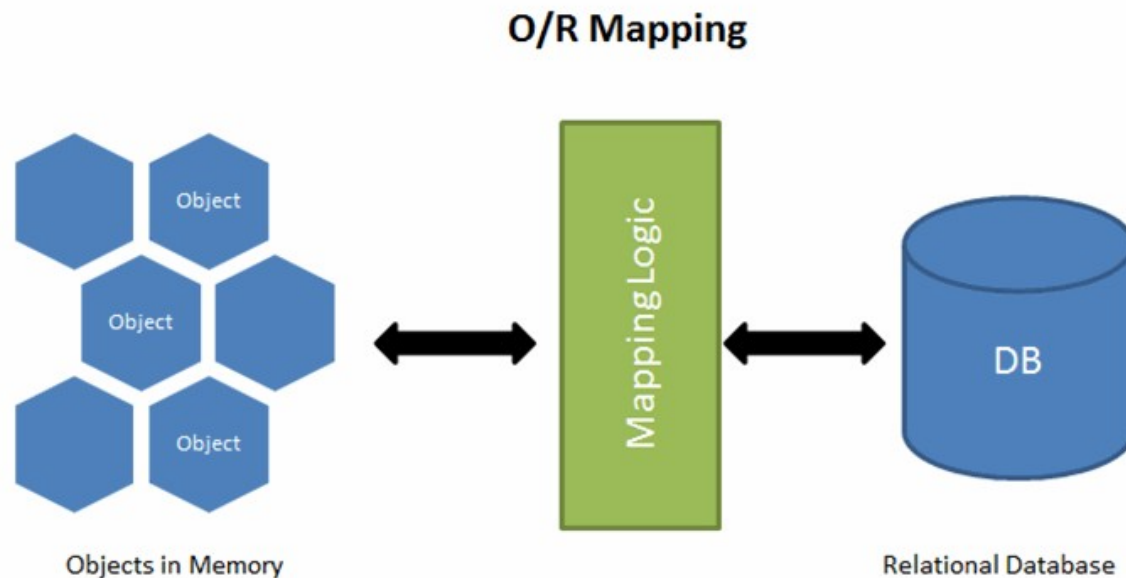
Esquema de banco de dados e ORM

- Nossa aplicação usará o ORM Prisma, e os modelos serão definidos no arquivo **prisma/schema.prisma**

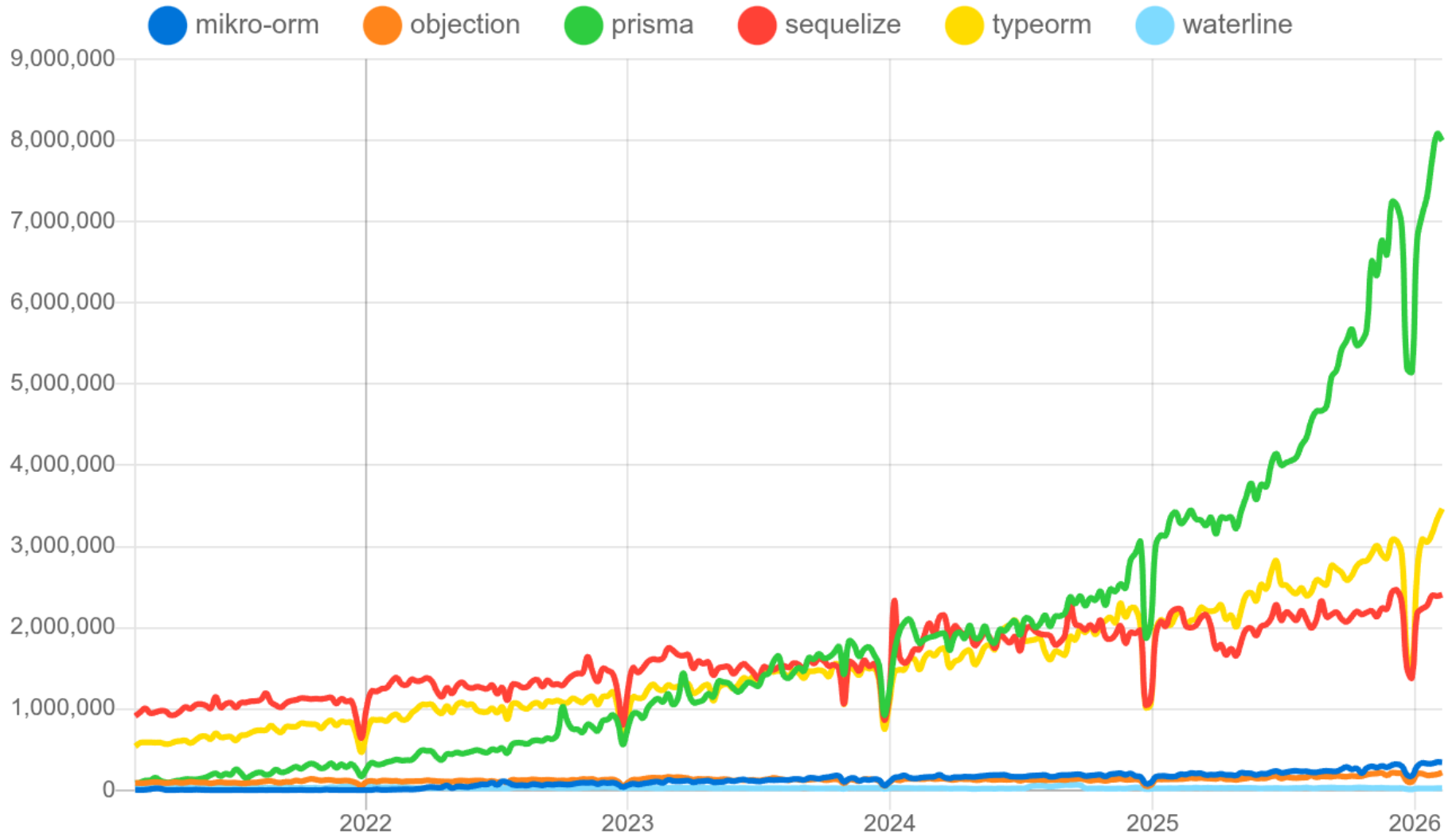


Object Relational Mapper - ORM

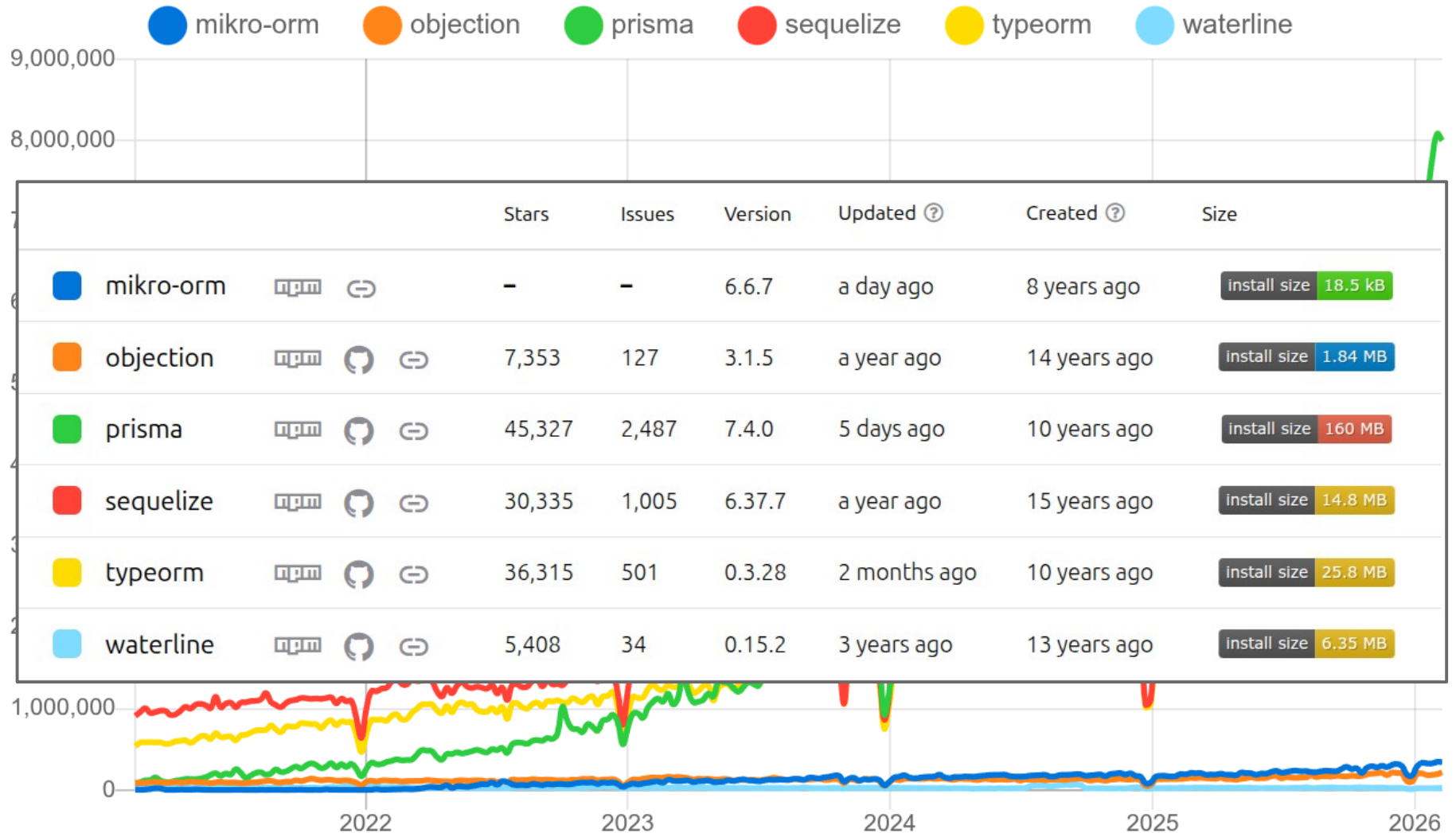
- ORM é uma técnica que permite **consultar e manipular dados** de um database usando o paradigma de orientação a objetos
- Desta forma, o acesso aos dados não é feito através da linguagem SQL, e sim através de objetos



Escolhendo o ORM



Escolhendo o ORM





- Para seguirmos, vamos criar um container Docker com o SGBD MySQL e outro com o PhpMyAdmin para acesso ao banco
- Para que o PhpMyAdmin acesse ao banco, precisamos criar uma rede Docker para que os dois containers se comuniquem

```
$ docker network create smartnotes-network
```

A screenshot of a terminal window with a dark background. The title bar at the top reads "david@coyote:~/dev/pw". The prompt is "david@coyote" in green, followed by the path "~/dev/pw" in blue. The command "docker network create smartnotes-network" is entered in white. The output is a long alphanumeric string: "888f6347a44da0435eb9e20e356ab421353b8fb1aaddb19b0a46912e09651b". The prompt "david@coyote" is shown again on the next line.

```
david@coyote ~/dev/pw $ docker network create smartnotes-network
888f6347a44da0435eb9e20e356ab421353b8fb1aaddb19b0a46912e09651b
david@coyote ~/dev/pw $
```



- Agora vamos criar um container Docker com o SGBD MySQL

```
$ docker run -d \  
  --name smartnotes-db \  
  --network smartnotes-network \  
  --restart unless-stopped \  
  -p 3316:3306 \  
  -e MYSQL_ROOT_PASSWORD=securepassword \  
  -e MYSQL_DATABASE=smartnotes \  
  -v mysql-game:/var/lib/mysql \  
  mysql:latest
```

- Os dados de acesso ao banco de dados criado são:
 - **Porta:** 3316
 - **Nome do banco:** game
 - **Host:** 127.0.0.1
 - **Senha do root:** senhasegura



- Agora vamos criar um container Docker com o SGBD MySQL

```
$ docker run -d \
```

No ambiente de desenvolvimento utilizamos o usuário root, pois o Prisma precisa criar bancos temporários para rodar as migrations e validar automaticamente o esquema do banco de dados

Em produção, por outro lado, deve-se utilizar um usuário sem privilégios administrativos, seguindo o princípio do menor privilégio possível

- Os dados de acesso ao banco de dados criado são:
 - **Porta:** 3316
 - **Nome do banco:** game
 - **Host:** 127.0.0.1
 - **Senha do root:** senhasegura



- Para acessar o banco de dados criado, vamos criar um container contendo uma instância do PhpMyAdmin

```
$ docker run -d \  
  --name smartnotes-phpmyadmin \  
  --network smartnotes-network \  
  --restart unless-stopped \  
  -e PMA_HOST=smartnotes-db \  
  -e PMA_PORT=3306 \  
  -e PMA_USER=root \  
  -e PMA_PASSWORD=securepassword \  
  -p 8081:80 \  
  phpmyadmin/phpmyadmin
```





localhost:8081 / smartno x +

localhost:8081

phpMyAdmin

Recente Favoritos

- Novo
- information_schema
- mysql
- performance_schema
- smartnotes
- sys

Servidor: smartnotes-db:3306

Bancos de dados SQL Status Contas de usuário Exportar Importar Mais

Configurações gerais

Collation de conexão do servidor MySQL: utf8mb4_unicode_ci

Mais configurações

Configurações do Tema

Idioma (Language) Português (Brasil) - Portuguese (Brazil)

Tema pmahomme Ver todos

Servidor de banco de dados

- Servidor: smartnotes-db via TCP/IP
- Tipo de servidor: MySQL
- Conexão do servidor: O SSL não está sendo usado
- Versão do servidor: 9.4.0 - MySQL Community Server - GPL
- Versão de protocolo: 10
- Usuário: root@172.29.0.3
- Charset do servidor: UTF-8 Unicode (utf8mb4)

Servidor web

- Apache/2.4.62 (Debian)
- Versão do cliente de banco de dados: libmysql - mysqlnd 8.2.27
- Extensão do PHP: mysqli curl mbstring sodium

Console



- Use os comandos abaixo para instalar o **Prisma** como dependência de desenvolvimento

```
$ npm install -D prisma  
$ npm install @prisma/client @prisma/adapter-mariadb
```

- Após isso, execute o comando abaixo para que o Prisma crie os arquivos iniciais de configuração

```
$ npx prisma init --datasource-provider mysql
```



```
david@coyote:~/dev/smartnotes/api
$ npx prisma init --datasource-provider mysql
Loaded Prisma config from prisma.config.ts
Initialized Prisma in your project
$ prisma
prisma/
  schema.prisma
  prisma.config.ts
warn Prisma would have added DATABASE_URL but it already exists in .env
warn You already have a .gitignore file. Don't forget to add .env in it to not c
ommit any private information.
$ npx prisma init --datasource-provider mysql
Next, set up your database:
1. Configure your DATABASE_URL in prisma.config.ts
2. Run prisma db pull to introspect your database.

Then, define your models in prisma/schema.prisma and run prisma migrate dev to a
pply your schema.
Learn more: https://pris.ly/getting-started

david@coyote ~/dev/smartnotes/api $
```



- O comando **prisma init** adiciona uma variável `DATABASE_URL` no arquivo **.env**, contendo uma string de conexão com o banco
- Será preciso editar o valor dessa variável conforme a realidade do banco de dados utilizado

```
DATABASE_URL=mysql://root:securepassword@127.0.0.1:3316/smartnotes
```

SGBD

usuário
do
banco

senha
do
usuário

host

porta

nome
do
banco



- O comando **prisma init** também cria um arquivo **prisma/schema.prisma**, onde serão criados os modelos da aplicação
- Nesse arquivo, é importante alterar o **provider** para **mysql**, conforme mostrado abaixo

```
generator client {  
  provider = "prisma-client-js",  
  output   = "../src/generated/prisma"  
}  
  
datasource db {  
  provider = "mysql"  
}
```



The screenshot shows the Visual Studio Code interface with the Extensions Marketplace open. The search bar contains 'prisma'. The 'Prisma' extension by prisma.io is highlighted. A blue speech bubble points to the extension's icon.

Prisma
Prisma [prisma.io](#) | 3,336,261 | ★★★★★ (30)
Adds syntax highlighting, formatting, auto-completion, jump-to-definition and linting for prisma files.

Uninstall Auto Update

Prisma VS Code

Para usar o Prisma, é altamente recomendado instalar a extensão **Prisma**, do **Visual Studio**

Prisma Import

Adicionando modelos

- Ainda no arquivo **prisma/schema.prisma**, vamos adicionar o model Note na nossa aplicação

```
model Note {  
  id String @id @default(uuid()) @db.Char(36)  
  
  title   String @db.VarChar(100)  
  content String @db.Text  
  
  createdAt DateTime @default(now()) @map("created_at")  
  updatedAt DateTime @updatedAt @map("updated_at")  
  
  @@map("notes")  
}
```

Adicionando modelos

- Após isso, usamos o comando **npx prisma migrate dev --name** para gerar a migração e criar as tabelas

```
david@coyote:~/dev/smartnotes/api
david@coyote ~/dev/smartnotes/api $ npx prisma migrate dev --name create-notes-table
Loaded Prisma config from prisma.config.ts.

Prisma config detected, skipping environment variable loading.
Prisma schema loaded from prisma/schema.prisma
Datasource "db": MySQL database "smartnotes" at "127.0.0.1:3316"

Applying migration `20260218121712_create_notes_table`

The following migration(s) have been created and applied from new schema changes:

prisma/migrations/
├─ 20260218121712_create_notes_table/
└─ migration.sql

Your database is now in sync with your schema.

✓ Generated Prisma Client (6.19.2) to ./src/generated/prisma in 27ms

david@coyote ~/dev/smartnotes/api $
```

Adicionando modelos

- Após isso, usamos o comando **npx prisma migrate dev --name** para gerar a migração e criar as tabelas

```
david@coyote:~/dev/smartnotes/api $ npx prisma migrate dev --name create-notes-table
Loaded Prisma config from prisma.config.ts.
```

Tabela	Ação
notes	Visualizar Estrutura Procurar Inserir Limpar Eliminar
_prisma_migrations	Visualizar Estrutura Procurar Inserir Limpar Eliminar

```
The following migration(s) have been created and applied from new schema changes:

prisma/migrations/
├─ 20260218121712_create_notes_table/
├─ migration.sql

Your database is now in sync with your schema.

✓ Generated Prisma Client (6.19.2) to ./src/generated/prisma in 27ms

david@coyote ~/dev/smartnotes/api $
```

Encontrando registros

- Para recuperar registros no banco de dados:

```
import { PrismaClient, User } from "@prisma/client";  
const prisma = new PrismaClient();  
const users: User[] = await prisma.user.findMany(criteria)
```

- Exemplos:

```
SELECT * FROM user WHERE nome = 'Carlos':
```

```
await prisma.user.findMany({ where: { nome: 'Carlos' } });
```

```
SELECT nome, idade FROM user WHERE nome = 'Carlos':
```

```
await prisma.user.findMany({  
  where: { nome: 'Carlos' },  
  select: { nome: true, idade: true }  
});
```

Encontrando registros

- Exemplos de consultas com o objeto **Sequelize.Op**

```
SELECT * FROM user WHERE idade < 30:
```

```
await prisma.user.findMany({ where: { idade: { lt: 30 } } });
```

```
SELECT * FROM user WHERE idade >= 21:
```

```
await prisma.user.findMany({ where: { idade: { gte: 21 } } });
```

```
SELECT * FROM user WHERE nome IN ('Carlos', 'Maria'):
```

```
await prisma.user.findMany({  
  where: { nome: { in: ['Carlos', 'Maria'] } }  
});
```

```
SELECT * FROM user WHERE nome NOT IN ('Carlos', 'Maria'):
```

```
await prisma.user.findMany({  
  where: { nome: { notIn: ['Carlos', 'Maria'] } }  
});
```

Encontrando registros

- Seleccionando usuários cujo nome possuem a string Carlos

```
SELECT * FROM user WHERE nome like '%Carlos%':
```

```
await prisma.user.findMany({  
  where: { nome: { contains: 'Carlos' } }  
});
```

- Seleccionando usuários cujo nome começam com Carlos

```
SELECT * FROM user WHERE nome like 'Carlos%':
```

```
await prisma.user.findMany({  
  where: { nome: { startsWith: 'Carlos' } }  
});
```

- Seleccionando usuários cujo nome terminam com Oliveira

```
SELECT * FROM user WHERE nome like '%Oliveira':
```

```
await prisma.user.findMany({  
  where: { nome: { endsWith: 'Oliveira' } }  
});
```

Paginação

- As cláusulas **skip** e **take** podem ser usadas em conjunto para contruir um sistema de paginação

- Selecionando os usuários 1-10 com idade maior que 20

```
SELECT * FROM user WHERE idade > 20 LIMIT 10 OFFSET 0;
```

```
await prisma.user.findMany({  
  where: { idade: { gt: 20 }},  
  skip: 0, take: 10  
});
```

- Selecionando os usuários 11-20 com idade maior que 20

```
SELECT * FROM user WHERE idade > 20 LIMIT 10 OFFSET 10;
```

```
await prisma.user.findMany({  
  where: { idade: { gt: 20 }},  
  skip: 10 , take: 10  
});
```

Ordenando Registros

- Os resultados recuperados podem ser ordenados de forma crescente (ASC) ou decrescente (DESC) por qualquer atributo
- Usuários >=18 ordenados pelo nome em ordem crescente

```
SELECT * FROM user WHERE idade >= 18 ORDER BY nome ASC;
```

```
await prisma.user.findMany({  
  where: { idade: { gte: 18 }}, orderBy: { nome, 'asc' }  
});
```

- Usuários >=18 ordenados pelo nome em ordem decrescente

```
SELECT * FROM user WHERE idade >= 18 ORDER BY nome DESC;
```

```
await prisma.user.findMany({  
  where: { idade: { gte: 18 }}, orderBy: { nome, 'desc' }  
});
```

Selecionando apenas um registro

- O comando **findMany()** retorna um array com todos os registros que atenderem os critérios da cláusula **where**
 - Note que, se apenas um registro atender aos critérios da cláusula **where**, será retornado um array com uma única posição
- Uma alternativa é o uso de **findFirst()**, que ao invés de recuperar um array, retorna apenas um objeto

```
SELECT * FROM user WHERE idade >= 18;
```

```
await prisma.user.findFirst({ where: { idade: { lte: 18 } } })
```

Criando novos registros

- Criação de novos registros no banco de dados:

```
await prisma.user.create(data: { initialValues } );
```

- Exemplo:

```
INSERT INTO user (nome, idade) VALUES ('Carlos', 26);
```

```
await prisma.user.create({ data: {nome: 'Carlos', idade: 16}})
```

Atualizando registros existentes

- Atualização de registros já existentes no banco de dados:

```
await prisma.user.update({values},{criteria});
```

- Exemplo:

```
UPDATE user SET idade=30 WHERE nome = 'Carlos';
```

```
await prisma.user.update(  
  { idade: 30 },  
  { where: { nome: 'Carlos' } }  
);
```

Apagando registros existentes

- Apagando registros no banco de dados:

```
await prisma.user.deleteMany(criteria);
```

- Exemplos:

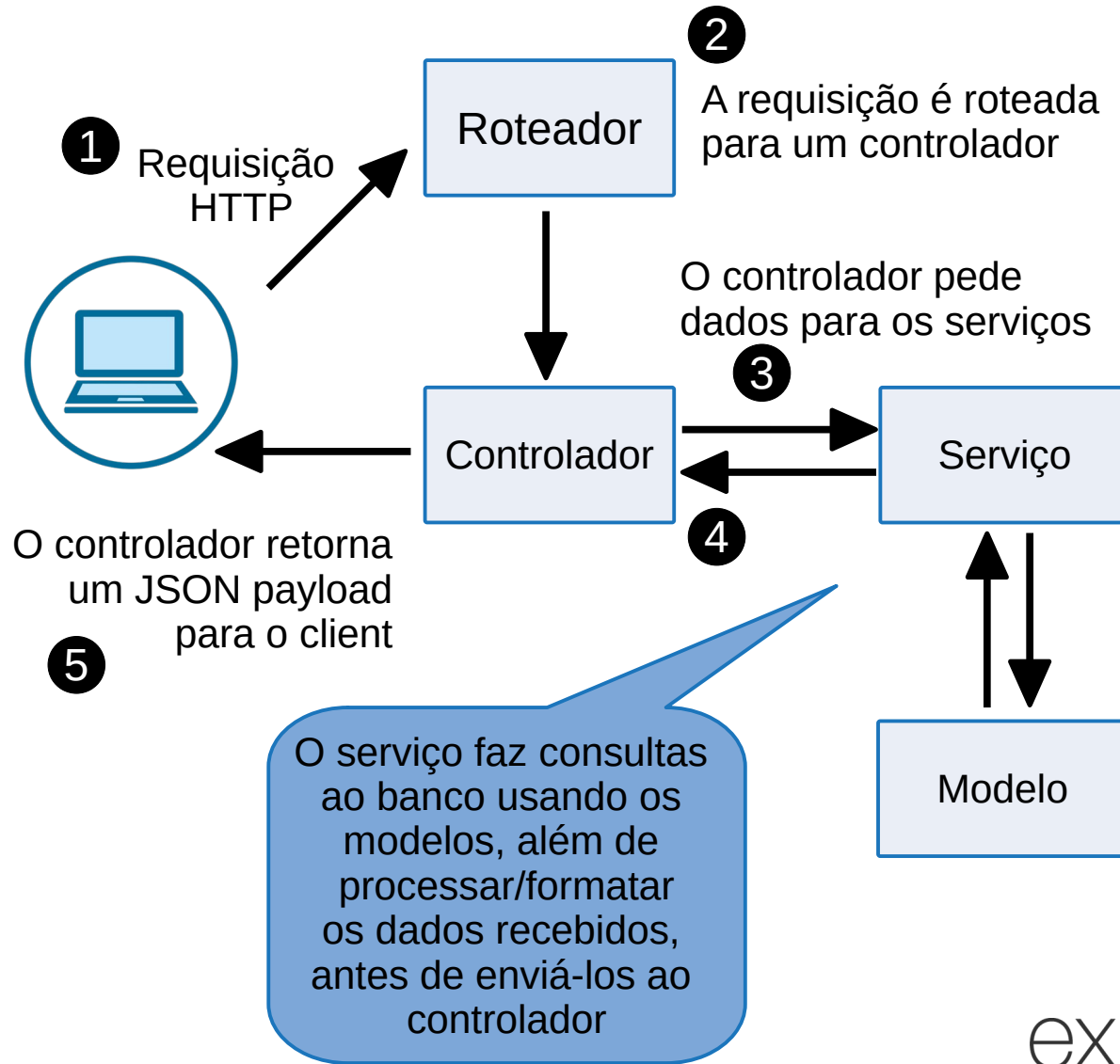
```
DELETE FROM user WHERE nome = 'Carlos';
```

```
await prisma.user.deleteMany({  
  where: { nome: 'Carlos' }  
});
```

```
DELETE FROM user WHERE id IN (3, 97);
```

```
await prisma.user.deleteMany({  
  where: { id: { in: [3, 97] } }  
});
```

Arquitetura da Aplicação



Camada de Roteamento

- Responsável por mapear as rotas HTTP (GET, POST, PUT, DELETE) para os respectivos controladores

```
// Arquivo src/resources/note/note.router.ts  
  
import { Router } from 'express';  
import noteController from './note.controller';  
  
const router = Router();  
  
// Note controller  
router.get('/', noteController.index);  
router.post('/', noteController.create);  
router.get('/:id', noteController.read);  
router.put('/:id', noteController.update);  
Router.delete('/:id', noteController.remove);  
  
export default router;
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/resources/note/note.service.ts

import { PrismaClient, Note } from "@generated/prisma/client";
import { PrismaMariaDb } from "@prisma/adapter-mariadb";
import { CreateNoteDto } from "../note.types";

const adapter = new PrismaMariaDb(process.env.DATABASE_URL!);
const prisma = new PrismaClient({ adapter });

export async function getAllNotes(): Promise<Note[]> {
  return await prisma.note.findMany();
}

export async function createNote(
  note: CreateNoteDto
): Promise<Note> {
  return await prisma.note.create({ data: note });
}
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/resources/note/note.service.ts
```

Além das funções mostradas, o serviço de note precisa ter funções como:

```
async function getNoteById(id: string): Promise<Note | null>
```

```
async function updateNote(  
  id: string,  
  note: UpdateNoteDto,  
) : Promise<Note>
```

```
async function deleteNote(id: string): Promise<Note>
```

```
export async function createNote(  
  note: CreateNoteDto
```

```
): Promise<Note> {  
  return await prisma.note.create({ data: note });  
}
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/resources/note/note.service.ts
```

Al Uma vantagem do uso de serviços é que suas funções podem ser utilizadas em outras partes da aplicação, diminuindo a réplica de códigos em vários arquivos.

```
async function updateNote(  
  id: string,  
  note: UpdateNoteDto,  
): Promise<Note> {  
  // ...  
}
```

```
async function deleteNote(id: string): Promise<Note>
```

```
export async function createNote(  
  note: CreateNoteDto  
): Promise<Note> {  
  return await prisma.note.create({ data: note });  
}
```

Camada de Serviços

- Os **serviços** têm como função orquestrar as regras de negócio e servir de intermediários entre controladores e modelos

```
// Arquivo src/resources/note/note.service.ts
```

Al Uma vantagem do uso de serviços é que suas funções podem ser utilizadas em outras partes da aplicação, diminuindo a réplica de códigos em vários arquivos.

Outra vantagem dos serviços é que, caso se queira mudar o ORM da aplicação, o esforço será muito menor. Isso porque eles serão os únicos arquivos que usam os recursos do ORM para recuperar, atualizar e criar dados.

```
async function deleteNote(id: string): Promise<Note>
```

```
export async function createNote(  
  note: CreateNoteDto
```

```
): Promise<Note> {  
  return await prisma.note.create({ data: note });  
}
```

Data Transfer Objects (DTO)

- Os arquivos **resources/**/*types.ts** possuem as **interfaces** e **types**, em especial os **DTOs**, usados dentro do resource
- DTO é uma interface ou type usado para representar os objetos de dados que são trocados entre a API e as aplicações client
 - Por exemplo, para criar uma nova nota, a aplicação cliente precisa enviar para a API os dados dessa nota, sendo o formato desses dados é definido através de um DTO

Data Transfer Objects (DTO)

- Os DTOs geralmente contêm um subconjunto dos atributos de um dado modelo, e para gerá-los podemos usar o comando Pick

```
// Arquivo src/resources/note/note.types.ts  
  
import { Note } from "@generated/prisma/client";  
  
export type CreateNoteDto = Pick<Note, "title" | "content">;  
export type UpdateNoteDto = Pick<Note, "title" | "content">;
```

Cria um novo type contendo apenas as propriedade title e content do modelo Note

Camada do Controlador

- Os controladores são responsáveis por **receptionar e responder** as respostas dos usuários

```
// Arquivo src/resources/note/note.controller.ts

import { Request, Response } from "express";
import { createNote } from "../note.service";
import { CreateNoteDto } from "../note.types";

async function create(req: Request, res: Response) {
  const note: CreateNoteDto = req.body;
  try {
    const newNote = await createNote(note);
    res.status(201).json(newNote);
  } catch (err) {
    res.status(500).json(err);
  }
}
```

Camada do Controlador

- Os controladores são responsáveis por **receptionar** e **responder** as respostas dos usuários

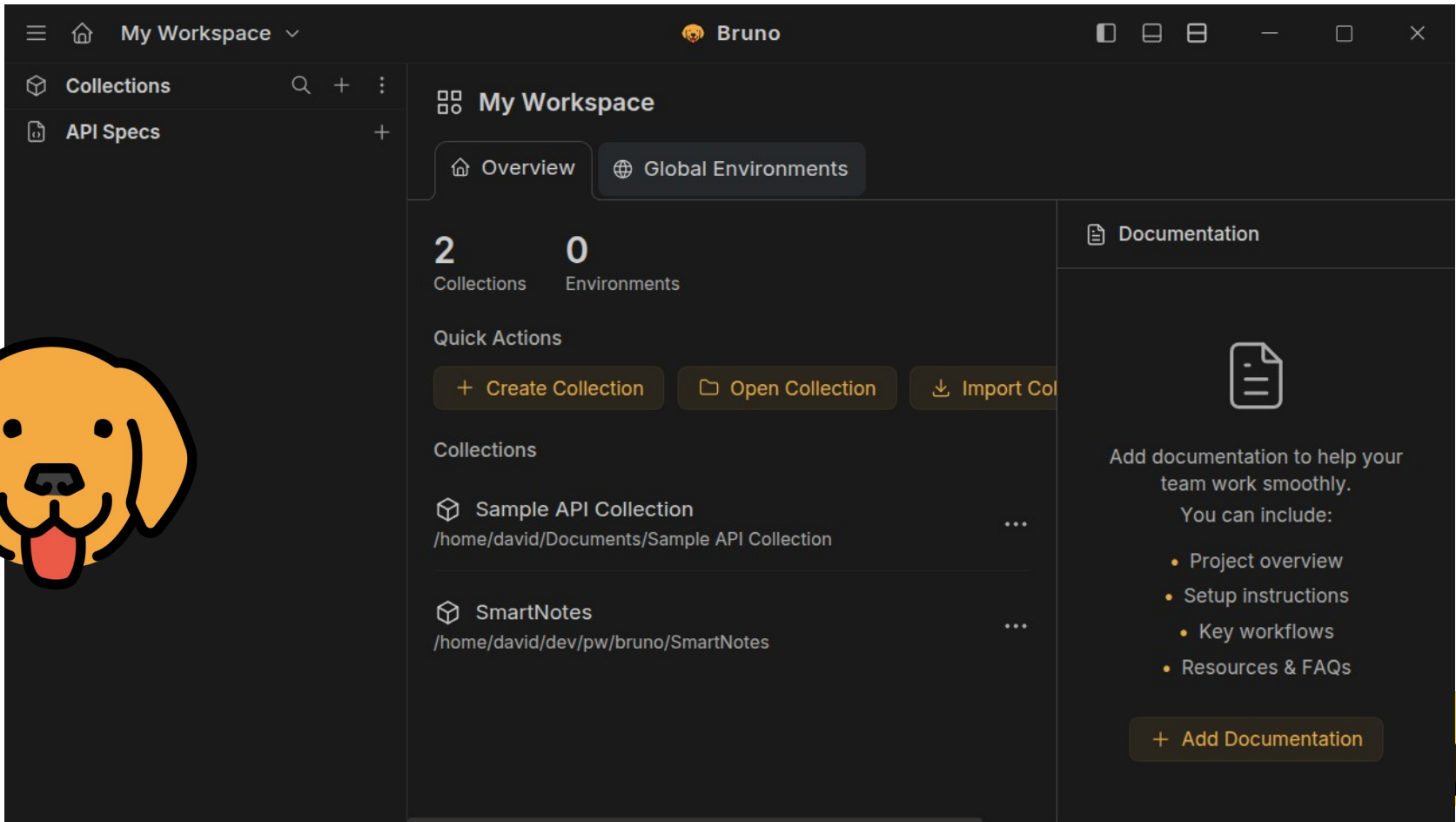
// Arquivo src/resources/note/note.controller.ts

É importante notar que erros na inserção de um registro disparam uma exceção no bloco try do Controlador, ocasionando um status 500 para a requisição.

```
async function create(req: Request, res: Response) {  
  const note: CreateNoteDto = req.body;  
  try {  
    const newNote = await createNote(note);  
    res.status(201).json(newNote);  
  } catch (err) {  
    res.status(500).json(err);  
  }  
}
```

HTTP Client Bruno

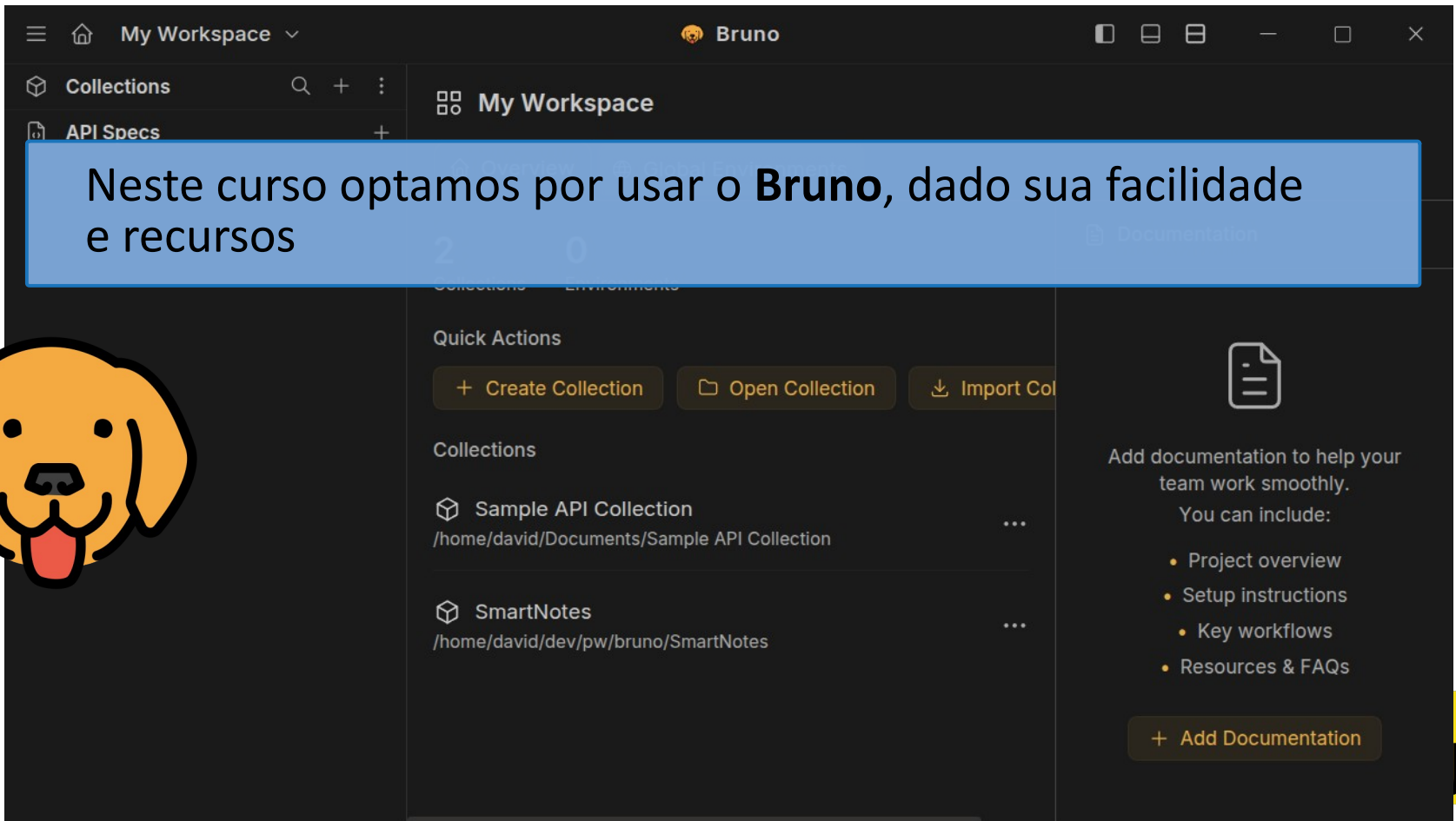
- Existem várias ferramentas para testar os endpoints de uma Api, como o **Insomnia**, o **Postman** e o **Bruno**



HTTP Client Bruno

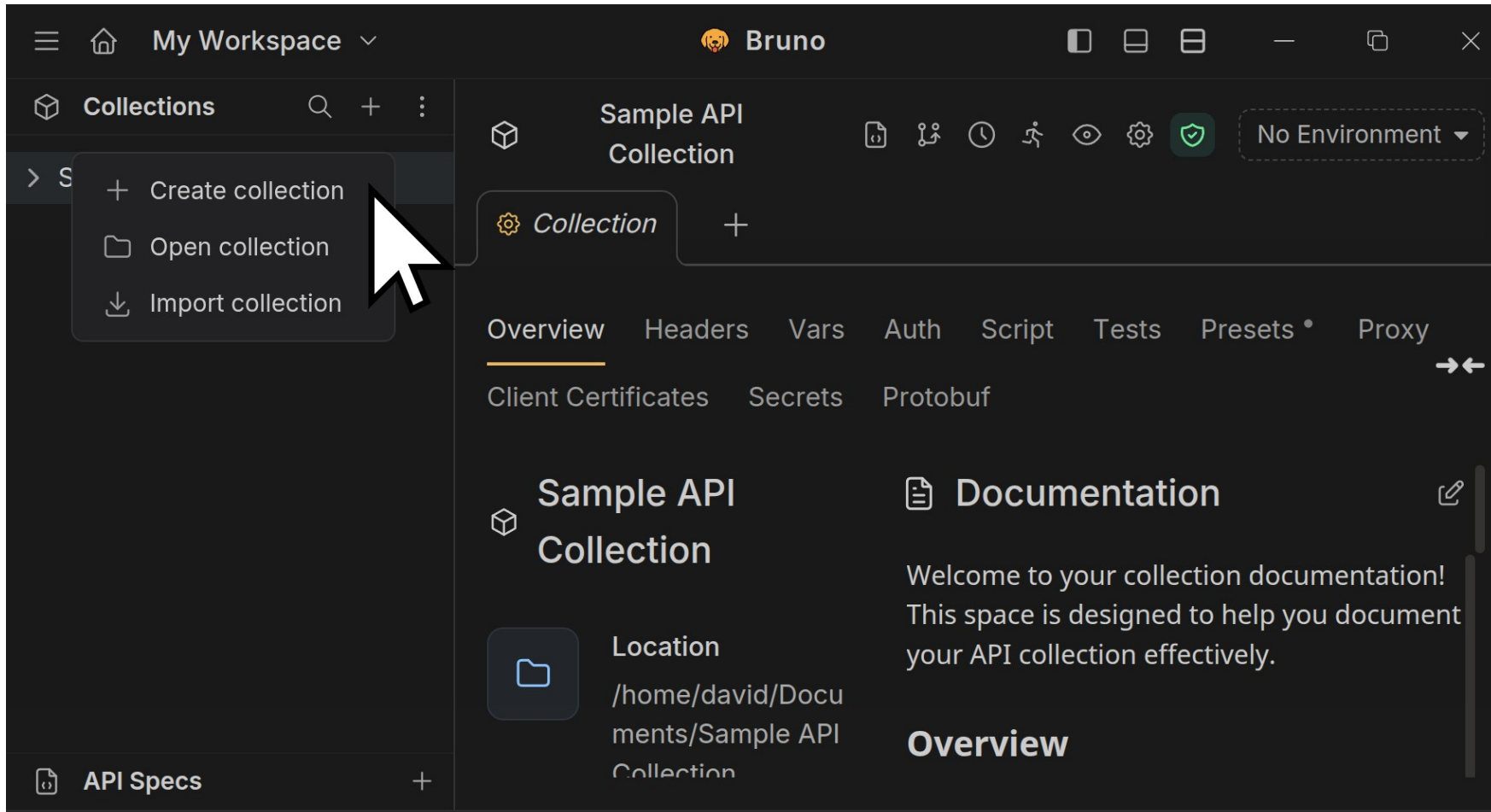
- Existem várias ferramentas para testar os endpoints de uma Api, como o **Insomnia**, o **Postman** e o **Bruno**

Neste curso optamos por usar o **Bruno**, dado sua facilidade e recursos



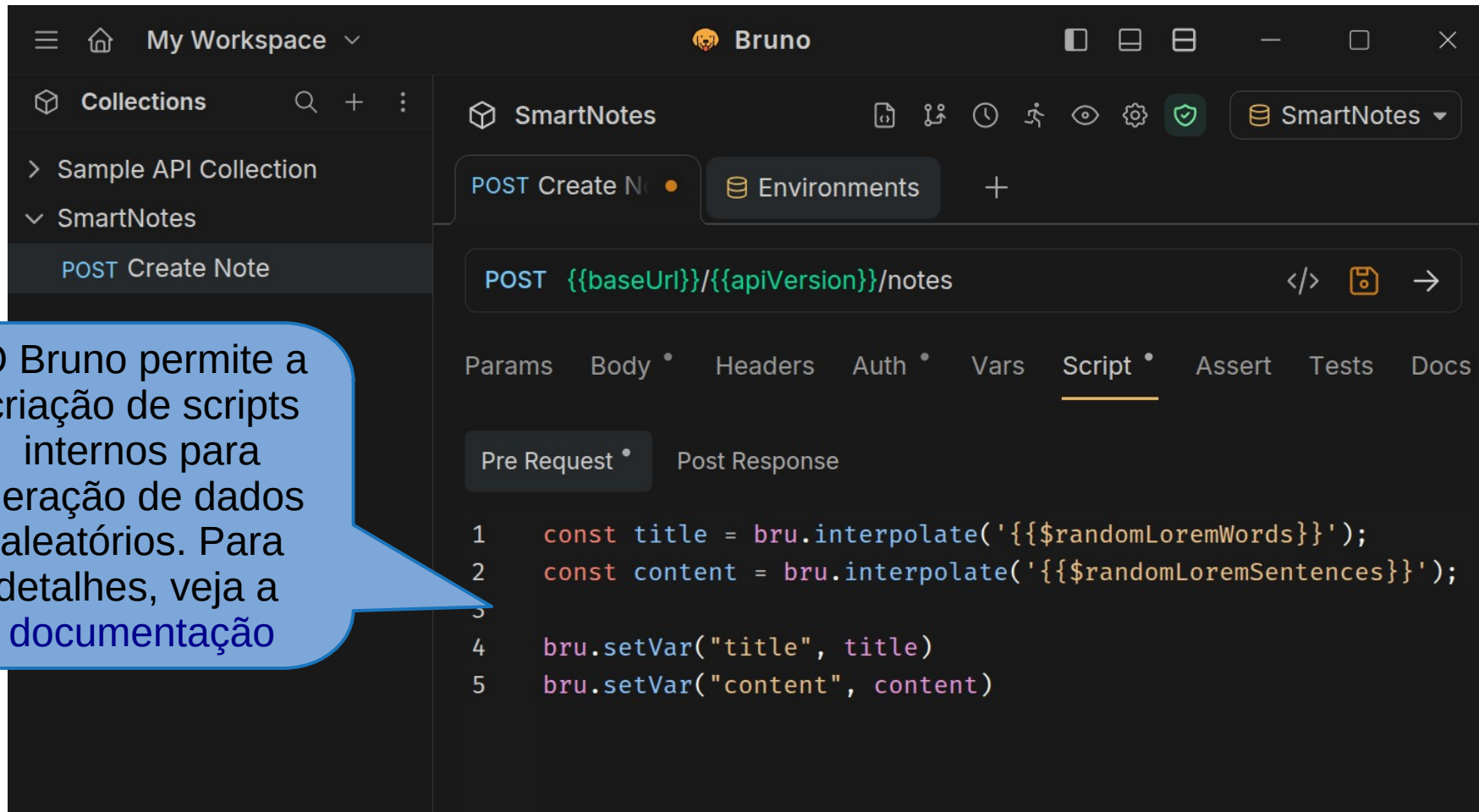
HTTP Client Bruno

- Para começar a usar o Bruno, crie uma coleção nova para a API sendo desenvolvida



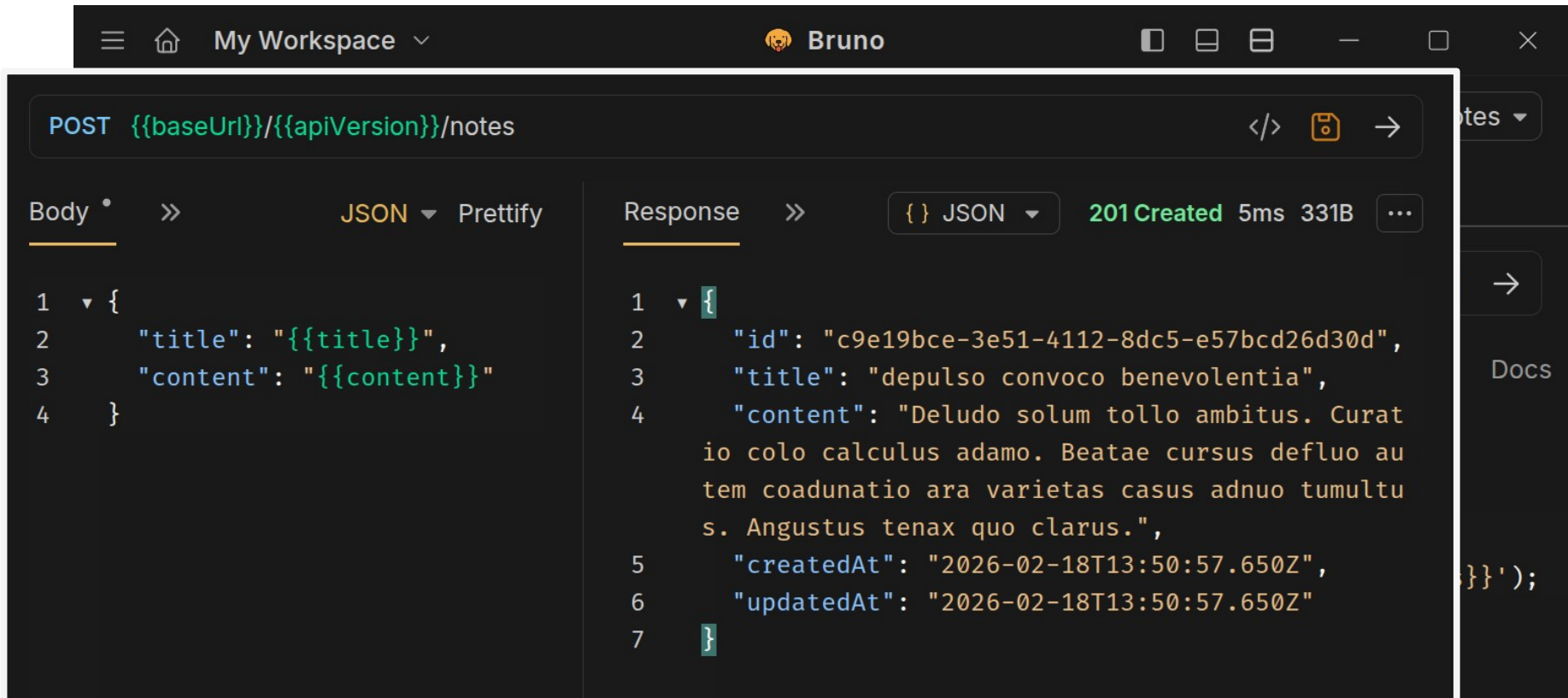
HTTP Client Bruno

- Para começar a usar o Bruno, crie uma coleção nova para a API sendo desenvolvida



HTTP Client Bruno

- Para começar a usar o Bruno, crie uma coleção nova para a API sendo desenvolvida



Sessões

- Através de sessões, podemos armazenar informações de estado (variáveis) no lado servidor
- Em vez do browser guardar um cookie por dado, ele guarda apenas um cookie contendo um **id de sessão (connect.sid)**

Requisições do Cliente

1 Get / HTTP1.1

3 Get /produto HTTP1.1
Cookie: connect.sid=6Bh

Respostas do Servidor

2 HTTP1.1 200 OK
Set-Cookie: connect.sid=6Bh
Conteúdo da requisição

4 HTTP1.1 200 OK
Conteúdo da requisição

Sessão 6Bh
Dados da sessão no servidor
user: Bruna
itens-carrinho: 3
start: 15:30



Sessões

- Para usarmos as sessões, precisamos instalar um módulo para geração de **valores únicos para os IDs** das sessões
- Uma opção é o módulo **uuid** – Universally Unique Identifier – que é uma implementação do UUID descrito na [RFC 4122](https://tools.ietf.org/html/rfc4122)

```
$ npm install uuid  
$ npm install -D @types/uuid
```

- Para habilitar o uso de sessões em sua aplicação, é necessário instalar o middleware **express-session**

```
$ npm install express-session  
$ npm install -D @types/express-session
```

sões

- Para usar o middle **connect.sid** vamos importar o módulo e adicioná-lo em nossa aplicação com o método **use**

evita o default

connect.sid

```
// Arquivo index.js
import session from 'express-session';
import { v4 as uuidv4 } from 'uuid';
...
app.use(session({
  name: 'sid',
  genid: () => uuidv4(),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 1000 * 60 * 60,
  },
}));
```

sões

- Para usar o middle **conne** portar o módulo e adicioná-lo em nosso **método use**

evita
defa

Os UUIDs são
usados para
gerar o
id de sessão

```
// Arquivo index.js
import session from 'express-session';
import { v4 as uuidv4 } from 'uuid';
...
app.use(session({
  name: 'sid',
  genid: () => uuidv4(),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 1000 * 60 * 60,
  },
}));
```

sões

- Para usar o middle adicioná-lo em noss

evita
defa
conne

Os UUIDs são
usados para
gerar o
id de sessão

Usado para adicionar
uma assinatura (similar ao
checksum) ao **id de sessão**
enviado para o usuário.
Quando o usuário devolve o
session.id, a assinatura é
usada para checar se o
session.id é válido. Usa uma
técnica chamada HMAC.

```
// Arquivo index.js
import session from 'express-session'
import { v4 as uuidv4 } from 'uuid'
...
app.use(session({
  name: 'sid',
  genid: () => uuidv4(),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 1000 * 60 * 60,
  },
}))
```

sões

- Para usar o middleware de sessão, você precisa adicioná-lo em

Quando **false**, a sessão do usuário não é salva a cada requisição, a menos que os dados da sessão tenham sido modificados durante a requisição

Usado para adicionar uma assinatura (similar ao checksum) ao **id de sessão** enviado para o usuário. Quando o usuário devolve o session.id, a assinatura é usada para checar se o session.id é válido. Usa uma técnica chamada HMAC.

```
// Arquivo index.js
import session from 'express-session'
import { v4 as uuidv4 } from 'uuid'

...
app.use(session({
  name: 'sid',
  genid: () => uuidv4(),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 1000 * 60 * 60,
  },
}))
```

sões

- Para usar o middleware de sessão, você precisa adicioná-lo em

Quando **false**, a sessão do usuário não é salva a cada requisição, a menos que os dados da sessão tenham sido modificados.

não cria sessão/cookie antes de precisar (ex: antes do login)

Usado para adicionar uma assinatura (similar ao checksum) ao **id de sessão** para o usuário. Quando o usuário devolve o `session.id`, a assinatura é usada para checar se o `session.id` é válido. Usa uma técnica chamada HMAC.

```
// Arquivo index.js
import session from 'express-session'
import { v4 as uuidv4 } from 'uuid'

...
app.use(session({
  name: 'sid',
  genid: () => uuidv4(),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 1000 * 60 * 60,
  },
}))
```

Sessões

- Para usar o middleware de sessão, você precisa adicioná-lo em seu arquivo de inicialização.

```
// Arquivo index.js
import session from 'express-session'
import { v4 as uuidv4 } from 'uuid'

...
app.use(session({
  name: 'sid',
  genid: () => uuidv4(),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 1000 * 60 * 60,
  },
}))
```

Quando **false**, a sessão do usuário não é salva a cada requisição, a menos que os dados da sessão tenham sido alterados.

renova a expiração a cada request

não cria sessão/cookie antes do login

Usado para adicionar uma assinatura (similar ao checksum) ao **id de sessão**.

Quando o usuário devolve o session.id, a assinatura é usada para checar se o session.id é válido. Usa uma técnica chamada HMAC.

sões

- Para usar o middleware de sessão, você precisa adicioná-lo em

Quando **false**, a sessão do usuário não é salva a cada requisição, a menos que os dados da sessão mudem.

renova a expiração a cada request

se não cria uma nova sessão/cookie a cada request (antes do login)

Usado para adicionar uma assinatura (similar ao checksum) ao **id de sessão**

Quando **true**, o id de sessão é usado para o usuário.

Quando **false**, o usuário devolve o `session.id`, a assinatura é adicionada para checar se o `session.id` é válido. Usa uma técnica chamada HMAC.

```
// Arquivo index.js
import session from 'express-session'
import { v4 as uuidv4 } from 'uuid'

...
app.use(session({
  name: 'sid',
  genid: () => uuidv4()
}))
```

Cookies

Search by domain

+ Add Cookie

x

localhost (2 cookies)

+ - ^

Name	Value	Path	Expires	Secure	HTTP Only	Actions
lang	pt-BR	/	Session		✓	
sid	s%3A18381522-4a47-448...	/	3/1/2026, 9:02:30 ...		✓	

```
}});
```

Same-Origin Policy (SOP)

- Dentro do browser, um script só pode fazer requisições para a mesma origem de onde foi carregado
- O nome dessa política é **Same-Origin Policy**, e sem ela qualquer site malicioso poderia fazer requisições autenticadas em nome do usuário usando os cookies dele
- No entanto, em casos de APIs, geralmente o frontend roda em uma porta separada do servidor

```
Browser (localhost:3001) → fetch("http://localhost:3333/v1/notes")
```

- Sem configuração, o browser bloqueia a resposta:

```
Access to fetch at 'http://localhost:3333/v1/notes' from origin 'http://localhost:3001' has been blocked by CORS policy.
```

Same-Origin Policy (SOP)

- CORS (Cross-Origin Resource Sharing) é o mecanismo que permite ao servidor dizer ao browser: "pode aceitar respostas dessa origem"
- Para isso, é necessário incluir um cabeçalho na resposta:

```
Access-Control-Allow-Origin: http://localhost:3001  
Access-Control-Allow-Credentials: true
```

- O browser lê esse cabeçalho e decide se entrega a resposta ao JavaScript ou bloqueia
- CORS é uma decisão do browser, ferramentas como curl, Bruno e Postman ignoram CORS completamente

Same-Origin Policy (SOP)

- Para instalar o CORS na aplicação express, use os comandos:

```
$ npm install cors  
$ npm install -D @types/cors
```

- Para permitir que apenas a aplicação Frontend (por exemplo, rodando no endereço `http://localhost:7777`) seja capaz de acessar a API, basta configurar o arquivo **src/index.ts**

```
import cors from "cors"  
...  
app.use(cors({  
  origin: ['http://localhost:7777'],  
  credentials: true,  
}));
```

Cadastro de Usuários

- No arquivo **prisma/schema.prisma**, vamos adicionar o modelo User e adaptar o modelo Note

```
model User {  
  id          String @id @default(uuid()) @db.Char(36)  
  email       String @unique @db.VarChar(100)  
  fullname    String @db.VarChar(100)  
  password    String @db.Char(60)  
  
  createdAt  DateTime @default(now()) @map("created_at") @db.DateTime(0)  
  updatedAt  DateTime @updatedAt @map("updated_at") @db.DateTime(0)  
  
  notes      Note[]  
  
  @@map("users")  
}
```

Cadastro de Usuários

- No arquivo **prisma/schema.prisma**, vamos adicionar o modelo User e adaptar o modelo Note

```
model Note {  
  id      String @id @default(uuid()) @db.Char(36)  
  userId  String @map("user_id") @db.Char(36)  
  
  title   String @db.VarChar(100)  
  content String @db.Text  
  
  createdAt DateTime @default(now()) @map("created_at") @db.DateTime(0)  
  updatedAt DateTime @updatedAt @map("updated_at") @db.DateTime(0)  
  
  user User @relation(fields: [userId], references: [id])  
  
  @@index([userId])  
  @@map("notes")  
}
```

Cadastro de Usuários

```
david@coyote:~/dev/pw $ npx prisma migrate dev --name create-user-model
Loaded Prisma config from prisma.config.ts.

Prisma config detected, skipping environment variable loading.
Prisma schema loaded from prisma/schema.prisma
Datasource "db": MySQL database "smartnotes" at "127.0.0.1:3316"

Applying migration 20260301121315_create_user_model

The following migration(s) have been created and applied from new schema changes:
prisma/migrations/
├─ 20260301121315_create_user_model/
├─ migration.sql
└─

Your database is now in sync with your schema.
user User @relation(fields: [userId], references: [id])
✓ Generated Prisma Client (6.19.2) to ./src/generated/prisma in 37ms

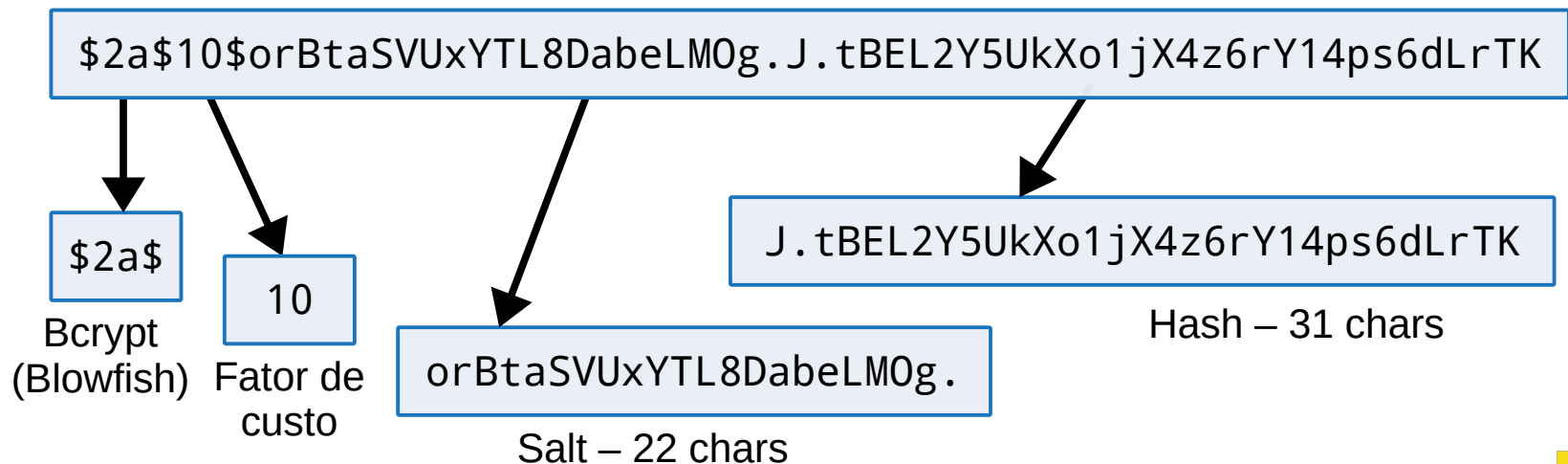
david@coyote:~/dev/pw $
```

O módulo bcrypt

- O módulo **bcrypt** é uma boa opção para geração de senhas, pois incorpora uma função hash à uma estratégia de salt

```
$ npm install bcryptjs  
$ npm install -D @types/bcryptjs
```

- O bcrypt é um **algoritmo de hash** usado para geração de senhas em sistemas como OpenBSD e algumas distribuições linux



O módulo bcrypt

- Para gerar uma senha com salt podemos usar o código abaixo, onde rounds é o número de rounds, e **senha** é a senha informada pelo usuário

```
const salt = await bcrypt.genSalt(rounds);  
const hash = await bcrypt.hash(senha, salt);
```

- Para verificar se uma senha está correta (no resource auth), podemos usar a função **compare** do bcrypt

```
const ok = await bcrypt.compare(senha, hash);
```

Senha
criptografada
(valor de hash
do código
anterior)

Resource Auth

- Para habilitar a autenticação de usuários, será criado um resource Auth contendo as funções de **signup**, **login** e **logout**

```
// Arquivo src/resources/auth/auth.router.ts  
  
import { Router } from 'express';  
import authController from '../auth.controller';  
  
const router = Router();  
  
router.post('/signup', authController.signup);  
router.post('/login', authController.login);  
router.post('/logout', authController.logout);  
  
export default router;
```

```
// Arquivo src/router/v1Router.ts  
  
import authRouter from '../resources/auth/auth.router';  
...  
router.use('/auth', authRouter);  
...
```

Resource Auth

- Para habilitar a autenticação de usuários, será criado um resource Auth controlador e **signup, login e logout**

A função **signup** será usada para que novos clientes criem seu próprio cadastro no app

```
// Arquivo src/resources/auth/auth.router.ts
import { Router } from 'express';
import authController from '../auth.controller';
const router = Router();
router.post('/signup', authController.signup);
router.post('/login', authController.login);
router.post('/logout', authController.logout);
export default router;
```

```
// Arquivo src/router/v1Router.ts
import authRouter from '../resources/auth/auth.router';
...
router.use('/auth', authRouter);
...
```

Resource Auth

- A função **signup** do controlador irá usar uma função **createUser**, da camada de serviço

```
// Arquivo src/resources/auth/auth.controller.ts

const signup = async (req: Request, res: Response) => {
  const usuario = req.body as SignUpDto;
  try {
    if (await getUserByEmail(usuario.email))
      return res
        .status(400)
        .json({ msg: "Email informado já está sendo usado" });
    const newUserario = await createUser(usuario);
    res.status(201).json(newUserario);
  } catch (e: any) {
    res.status(500).json(e.errors);
  }
};
```

Usa o
bcryptjs para
criptografar as
senhas

Resource Auth

- A função **login** do controlador Auth irá usar uma função **checkAuth** da camada de serviço de Auth

```
// Arquivo src/resources/auth/auth.service.ts
export const checkAuth = async (
  credenciais: LoginDto,
): Promise<User | null> => {
  const { email, password } = credenciais;
  const user = await prisma.user.findFirst({
    where: { email }
  });
  if (!user) return null;
  const ok = await compare(password, user.password);
  return ok ? user : null;
};
```

Usa o **bcryptjs** para verificar a senha digitada pelo usuário no login

Resource Auth

- A função **login** do controlador Auth irá usar uma função **checkAuth** da camada de serviço de Auth
- Usamos o D_HASH quando o usuário não existe, para que o bcrypt sempre execute e o tempo de resposta seja sempre igual

```
const D_HASH =  
  "$2b$10$invalidhashfortimingattack000000000000000000000000";  
  
export const checkAuth = async (  
  credenciais: LoginDto,  
) : Promise<User | null> => {  
  const { email, password } = credenciais;  
  const user = await prisma.user.findFirst({  
    where: { email }  
  });  
  const ok = await compare(password, user?.password ?? D_HASH);  
  return user && ok ? user : null;  
};
```

Resource Auth

- Ao efetuar o login, criamos as variáveis de sessão **uid** e **tipoUsuarioid** serão criadas

```
// Arquivo src/resources/auth.controller.ts

const login = async (req: Request, res: Response) => {
  const { email, password } = req.body as LoginDto;
  try {
    const user = await checkAuth({ email, password });
    if (!user)
      return res.status(401).json({
        msg: 'Email e/ou senha incorretos'
      });
    req.session.uid = user.id;
    res.status(200).json({ msg: 'Usuário autenticado' });
  } catch (e) {
    res.status(500).json(e);
  }
}
```



Variável de sessão

Resource Auth

- Para que as variáveis de sessão funcionem, é preciso adicionar tais atributos à interface **SessionData** de **express-session**
- Para isso, adicionamos o seguinte código no começo (após a importação dos pacotes) do arquivo **src/index.ts**

```
declare module "express-session" {  
  interface SessionData {  
    uid: string;  
  }  
}
```

- p
- ta
- p
- in

ar

The screenshot displays the Bruno API client interface. The top bar shows 'My Workspace' and the Bruno logo. The left sidebar contains a 'Collections' panel with a tree view: 'Sample API Collection' (expanded), 'SmartNotes' (expanded), 'Note', 'Language', 'Auth' (expanded), 'POST Login' (selected), 'POST SignUp', and 'POST Logout'. The main panel shows the 'POST Login' request details. The URL is 'POST {{baseUrl}}/{{apiVersion}}/auth/login'. The 'Body' tab is active, showing a JSON object:

```
{ 1: { 2: "email": "Jayme_Hagenes62@yahoo.com", 3: "password": "A123456a$", 4: }
```

. The 'Response' tab is also active, showing a JSON object:

```
{ 1: { 2: "msg": "Usuário autenticado", 3: }
```

. The status bar at the bottom indicates '200 OK 77ms 30B'. The bottom of the interface includes a status bar with 'Try Bruno Ultimate', a search bar, 'Cookies', 'Dev Tools', and the version 'v3.1.4'.

My Workspace

Collections

- Sample API Collection
- SmartNotes
 - Note
 - Language
 - Auth

SmartNotes

POST Login

POST {{baseUrl}}/{{apiVersion}}/auth/login

Params Body JSON Prettify

Cookies

Search by domain Add Cookie

localhost (2 cookies)

Name	Value	Path	Expires	Secure	HTTP Only	Actions
lang	pt-BR	/	Session		✓	
sid	s%3Af3dd0d43-5cd4-45...	/	3/10/2026, 7:26:47...		✓	

API Specs

```
1 {
2   "msg": "Usuário autenticado"
3 }
```

Try Bruno Ultimate Search Cookies Dev Tools v3.1.4

Mecanismo de Autorização

- Um middleware **isAuth** pode ser criado para que apenas usuários logados possam acessar as rotas do resource note

```
import { NextFunction, Request, Response } from "express";

function isAuth(
  req: Request,
  res: Response,
  next: NextFunction
) {
  if (req.session.userId) next();
  else res.status(401).json({
    msg: "Usuário não autenticado"
  });
}

export default isAuth;
```

Mecanismo de Autorização

- Uma vez que o middleware **isAdmin** seja criado, podemos impedir que usuários não logados acessem as rotas das notas

```
// Arquivo src/resources/note/note.router.ts  
  
import { Router } from "express";  
import noteController from "../note.controller";  
import schema from "../note.schema";  
import validate from "../../middlewares/validateBody";  
import isAuthenticated from "../../middlewares/isAuth";  
  
const router = Router();  
  
router.use(isAuthenticated);  
  
router.get("/", noteController.index);  
router.post("/", validate(schema), noteController.create);  
router.get("/:id", noteController.read);  
router.put("/:id", validate(schema), noteController.update);  
router.delete("/:id", noteController.remove);  
  
export default router;
```

Mecanismo de Autorização

- Uma vez que o middleware **isAdmin** seja criado, podemos impedir que usuários não logados acessem as rotas das notas

```
feat/router  
chore/setup-orm  
feat/note-crud  
feat/note-joi  
chore/session  
feat/authn  
feat/authz  
chore/cors
```